KENNESAW STATE
UNIVERSITY

CS 4277
Video Understanding

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

Content

- From 2D to 3D CNN for video understanding
- Optical Flow
- RNN based video understanding
- Attention based video understanding
- Applications

Recall: (2D) Image classification

This image by Nikita is licensed under CC-BY 2.0

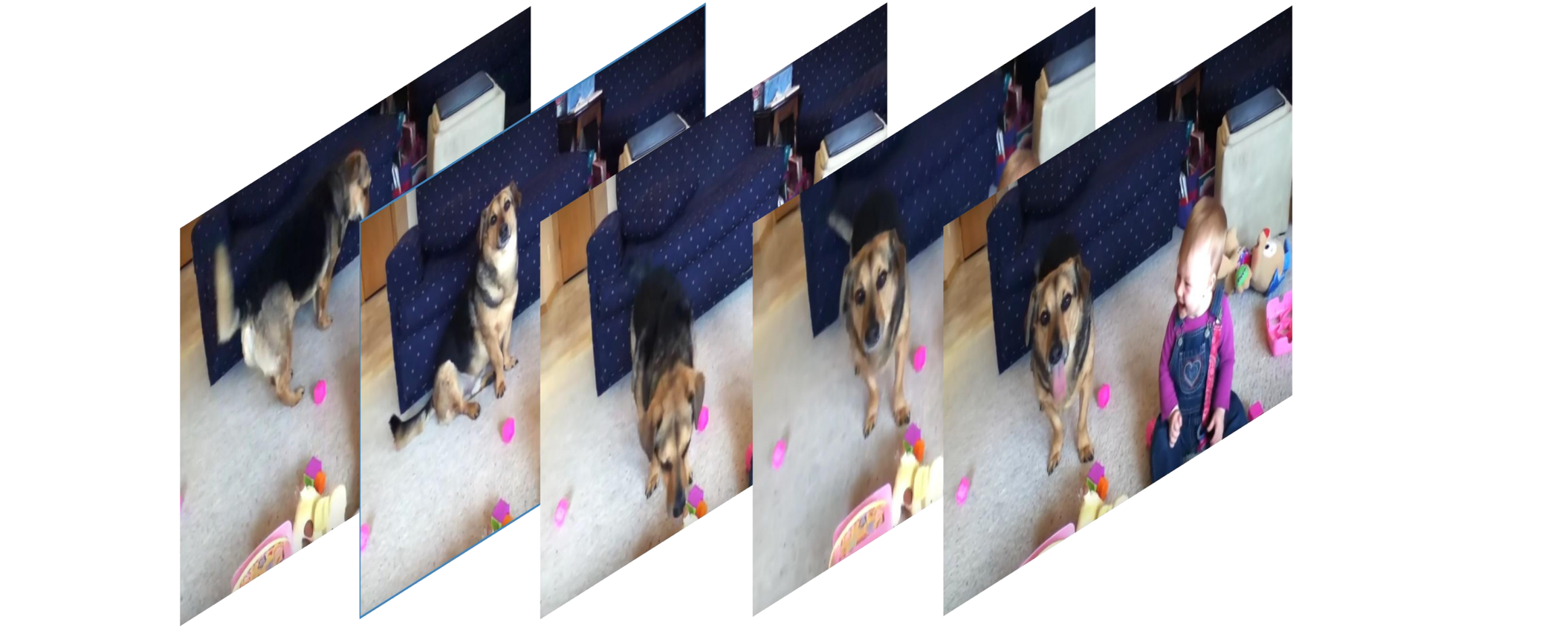
(assume given a set of possible labels)
{dog, cat, truck, plane, ...}

→ cat

Video = 2D + Time

A video is a sequence of images 4D tensor: T x 3 x H x W

(or 3 x T x H x W)



Example task: Video Classification

Input video:

T x 3 x H x W

Swimming

Running

Jumping

Eating

Standing

Example task: Video Classification

Images: Recognize objects

Dog
Cat
Fish
Truck

Videos: Recognize actions

Swimming
Running
Jumping
Eating
Standing

Problems: Videos are big!

Input video:

T x 3 x H x W

- Videos are ~30 frames per second (fps)
- Size of uncompressed video (3 bytes per pixel):
- SD (640 x 480): ~1.5 GB per minute
- HD (1920 x 1080): ~10 GB per minute
- Solution: Train on short clips: low fps and low spatial resolution
- e.g. T = 16, H=W=112
- (3.2 seconds at 5 fps, 588 KB)

Training on Clips

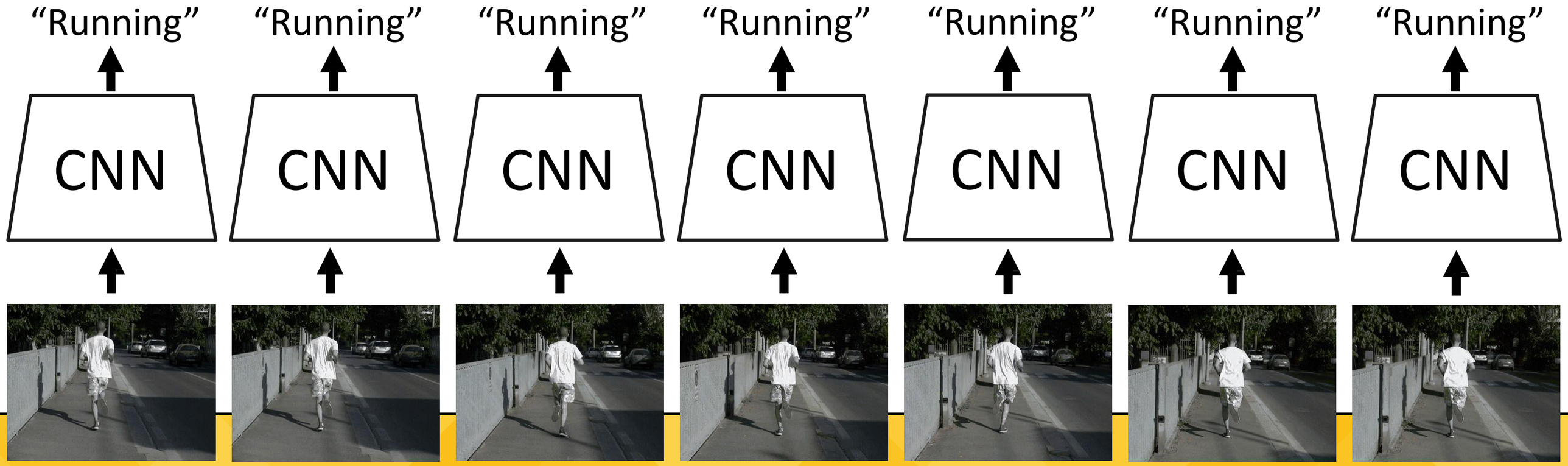
- Raw video: Long, high FPS

- Training: Train model to classify short clips with low FPS

- Testing: Run model on different clips, average predictions

Video Classification: Single-Frame CNN

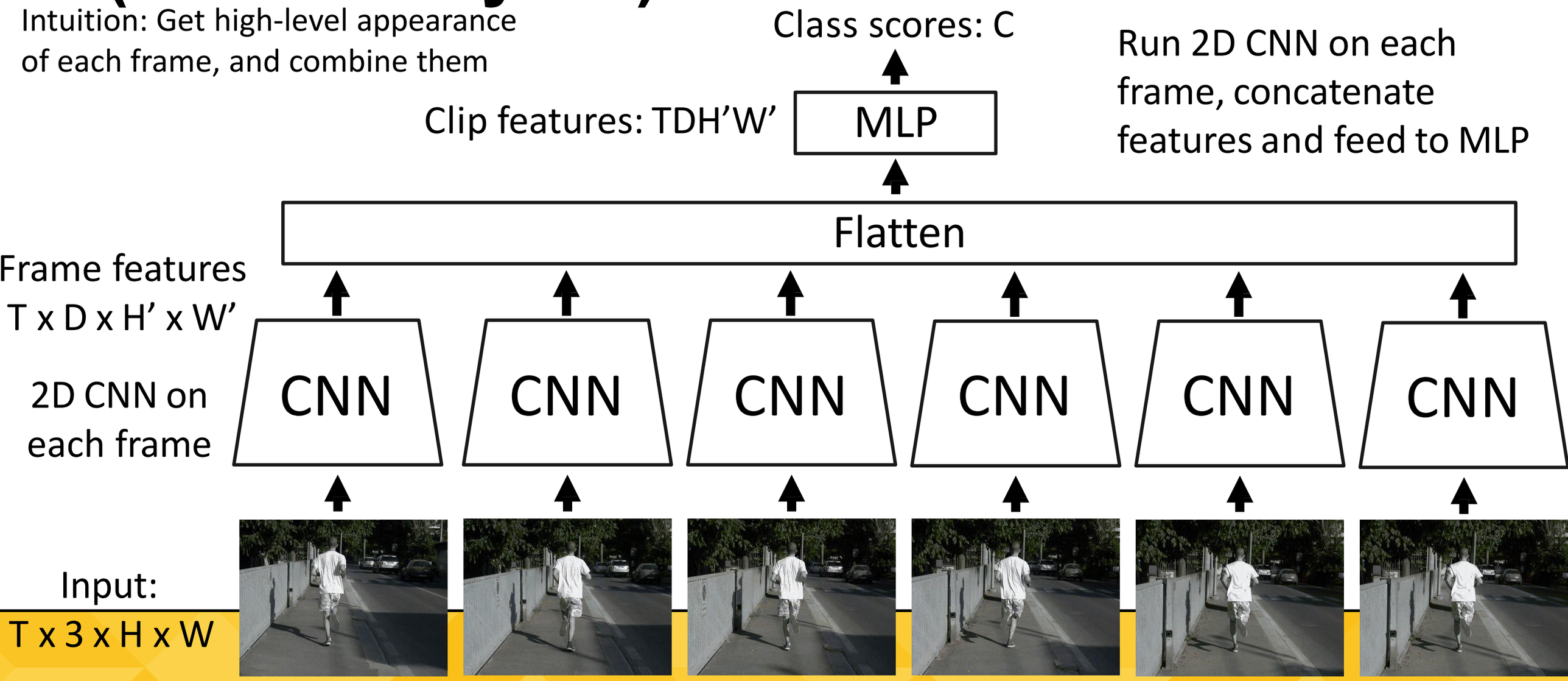
- Simple idea: train normal 2D CNN to classify video frames independently! (Average predicted probs at test-time)
- Often a very strong baseline for video classification



Video Classification: Late Fusion (with FC layers)

Intuition: Get high-level appearance of each frame, and combine them

Run 2D CNN on each frame, concatenate features and feed to MLP

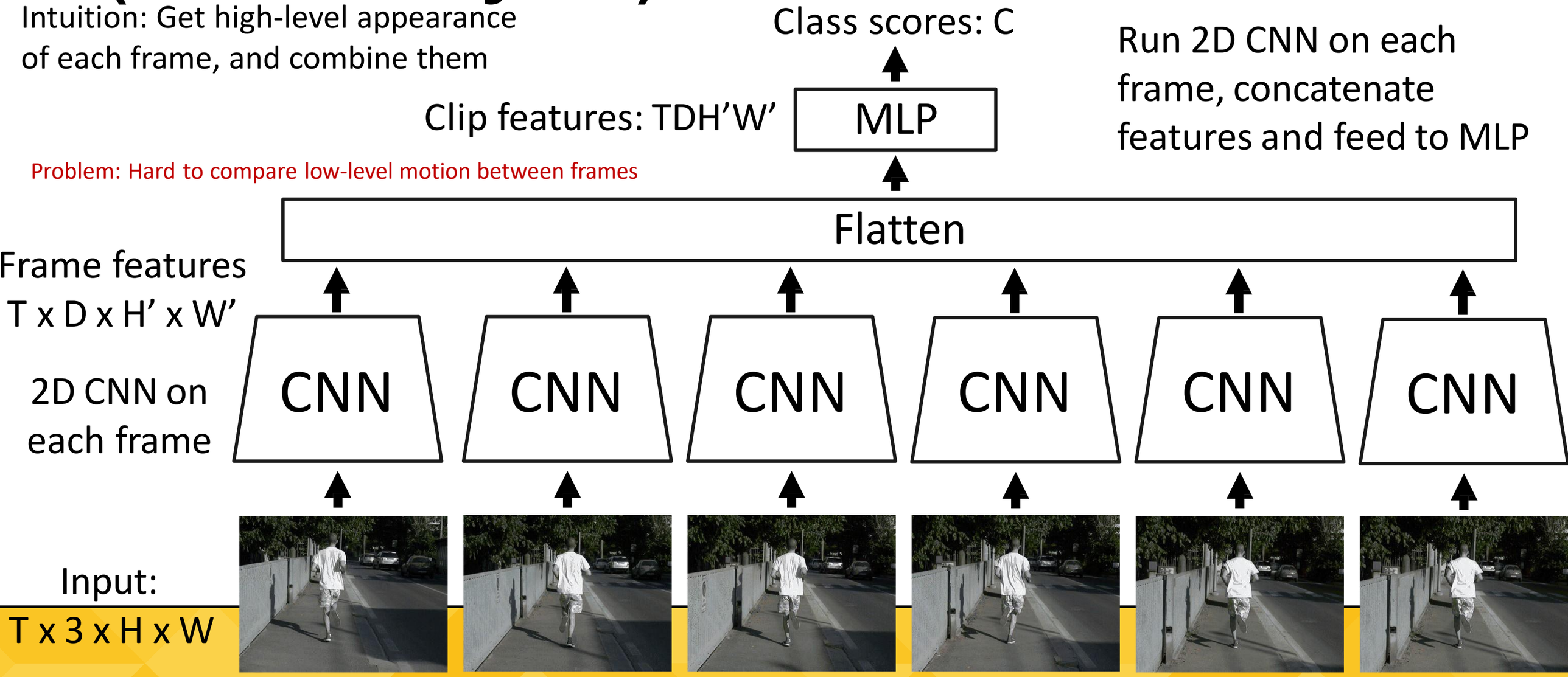


Video Classification: Late Fusion (with FC layers)

Intuition: Get high-level appearance of each frame, and combine them

Problem: Hard to compare low-level motion between frames

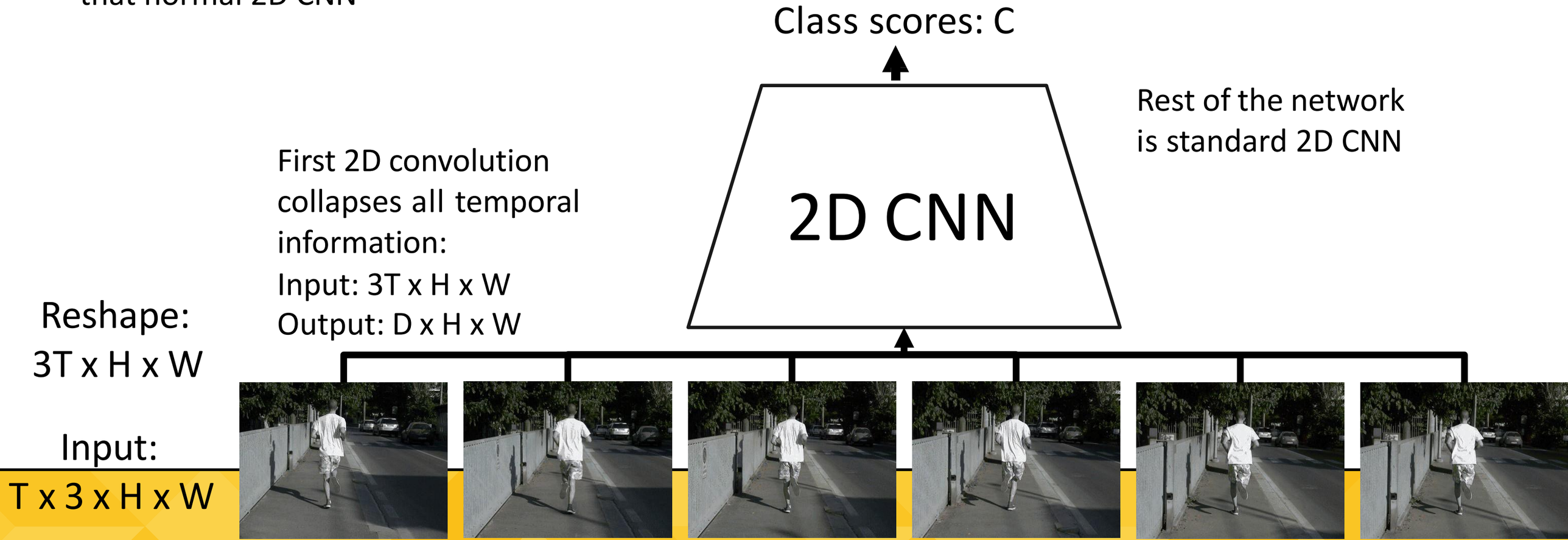
Run 2D CNN on each frame, concatenate features and feed to MLP



Video Classification: Early Fusion

Intuition: Compare frames with very first conv layer, after that normal 2D CNN

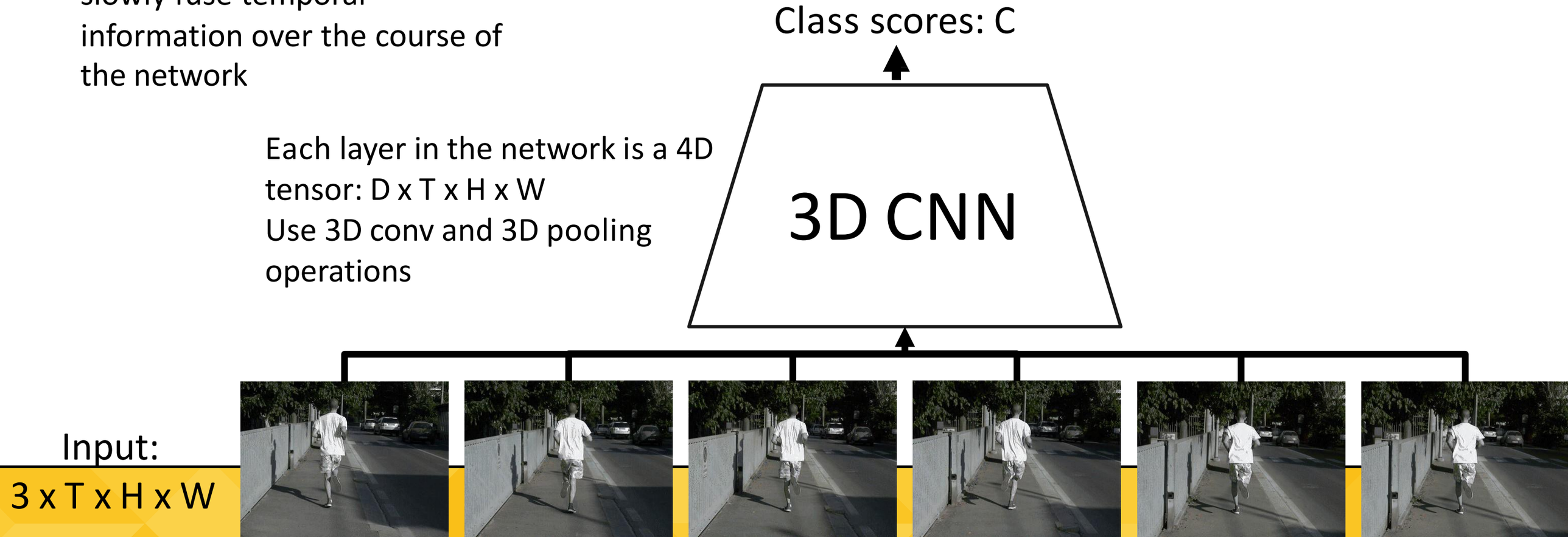
Problem: One layer of temporal processing may not be enough!



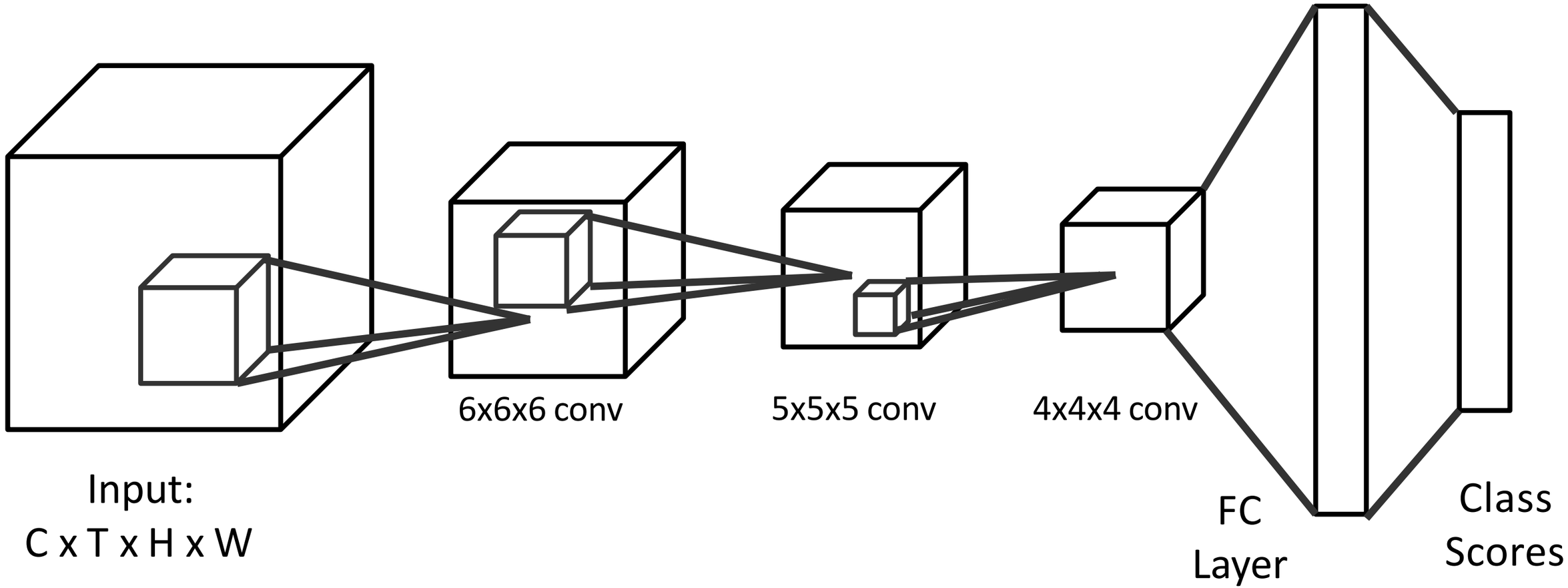
Video Classification: 3D CNN

Intuition: Use 3D versions of convolution and pooling to slowly fuse temporal information over the course of the network

Each layer in the network is a 4D tensor: D x T x H x W
Use 3D conv and 3D pooling operations



3D Convolution

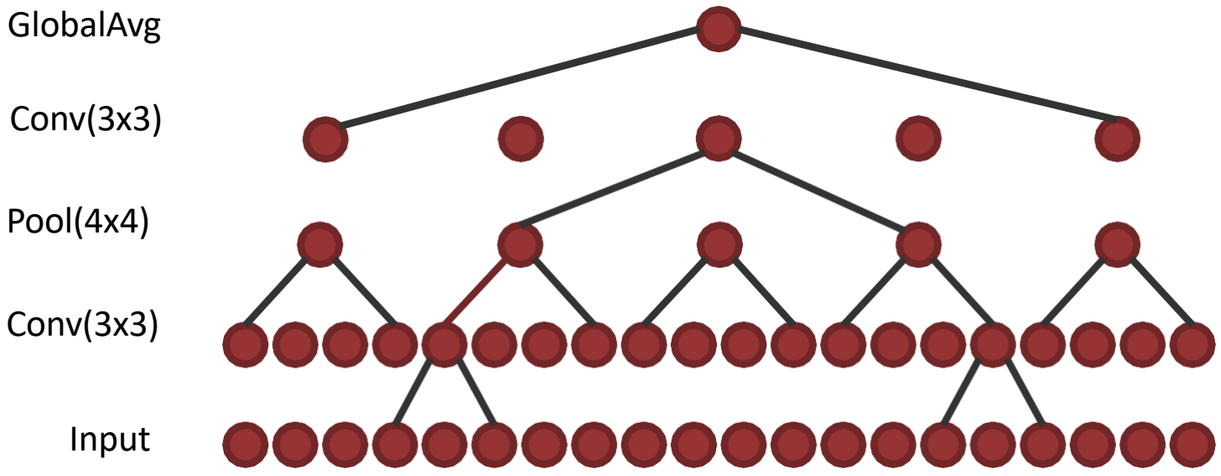


Early Fusion vs Late Fusion vs 3D CNN

Late Fusion

Layer	Size **(C x T x H x W)**	Receptive Field **(T x H x W)**
Input	3 x 20 x 64 x 64	
Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3
Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6
Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14
GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64

Build slowly in space,
All-at-once in time at end



Early Fusion vs Late Fusion vs 3D CNN

	Layer	Size **(C x T x H x W)**	Receptive Field **(T x H x W)**	
Late Fusion	Input	3 x 20 x 64 x 64		
	Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3	Build slowly in space, All-at-once in time at end
	Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6	
	Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64	
Early Fusion	Input	3 x 20 x 64 x 64		
	Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3	Build slowly in space, All-at-once in time at start
	Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6	
	Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64	

Early Fusion vs Late Fusion vs 3D CNN

	Layer	Size **(C x T x H x W)**	Receptive Field **(T x H x W)**	
Late Fusion	Input	3 x 20 x 64 x 64		Build slowly in space, All-at-once in time at end
	Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3	
	Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6	
	Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64	
Early Fusion	Input	3 x 20 x 64 x 64		Build slowly in space, All-at-once in time at start
	Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3	
	Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6	
	Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64	
3D CNN	Input	3 x 20 x 64 x 64		Build slowly in space, Build slowly in time ”Slow Fusion”
	Conv3D(3x3x3, 3->12)	12 x 20 x 64 x 64	3 x 3 x 3	
	Pool3D(4x4x4)	12 x 5 x 16 x 16	6 x 6 x 6	
	Conv3D(3x3x3, 12->24)	24 x 5 x 16 x 16	14 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64	

Early Fusion vs Late Fusion vs 3D CNN

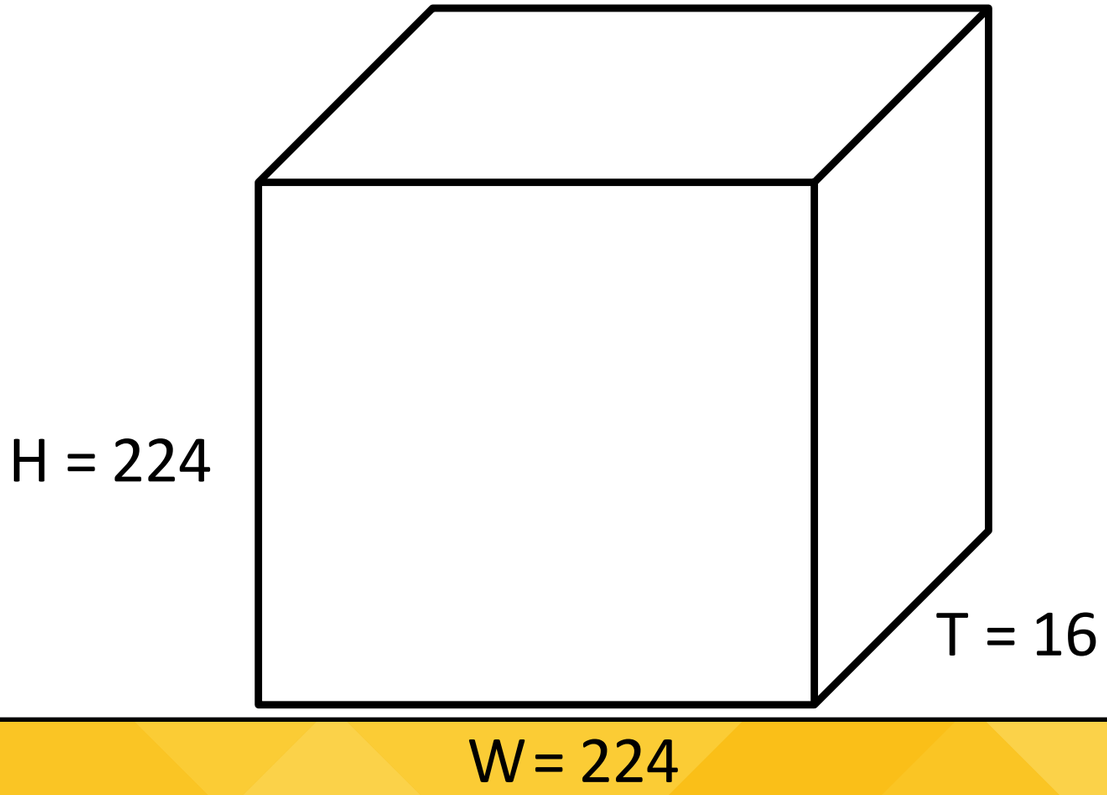
	Layer	Size **(C x T x H x W)**	Receptive Field **(T x H x W)**	What is the difference?
Late Fusion	Input	3 x 20 x 64 x 64		
	Conv2D(3x3, 3->12)	12 x 20 x 64 x 64	1 x 3 x 3	Build slowly in space, All-at-once in time at end
	Pool2D(4x4)	12 x 20 x 16 x 16	1 x 6 x 6	
	Conv2D(3x3, 12->24)	24 x 20 x 16 x 16	1 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1 x 1	20 x 64 x 64	
Early Fusion	Input	3 x 20 x 64 x 64		
	Conv2D(3x3, 3*20->12)	12 x 64 x 64	20 x 3 x 3	
	Pool2D(4x4)	12 x 16 x 16	20 x 6 x 6	Build slowly in space, All-at-once in time at start
	Conv2D(3x3, 12->24)	24 x 16 x 16	20 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64	
3D CNN	Input	3 x 20 x 64 x 64		
	Conv3D(3x3x3, 3->12)	12 x 20 x 64 x 64	3 x 3 x 3	
	Pool3D(4x4x4)	12 x 5 x 16 x 16	6 x 6 x 6	Build slowly in space, Build slowly in time ”Slow Fusion”
	Conv3D(3x3x3, 12->24)	24 x 5 x 16 x 16	14 x 14 x 14	
	GlobalAvgPool	24 x 1 x 1	20 x 64 x 64	

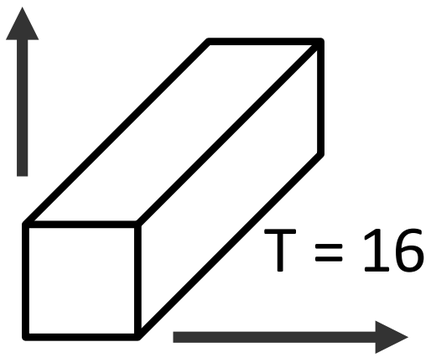
2D Conv (Early Fusion) vs 3D Conv (3D CNN)

Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

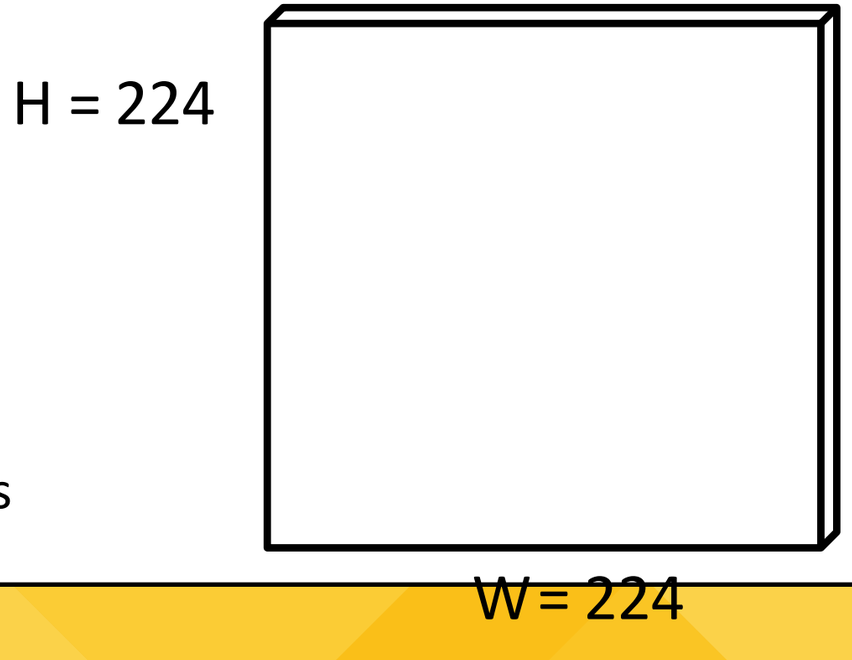
Weight:
C_{out} x C_{in} x T x 3 x 3
Slide over x and y

Output:
C_{out} x H x W
2D grid with C_{out} – dim feat at each point





C_{out} different filters



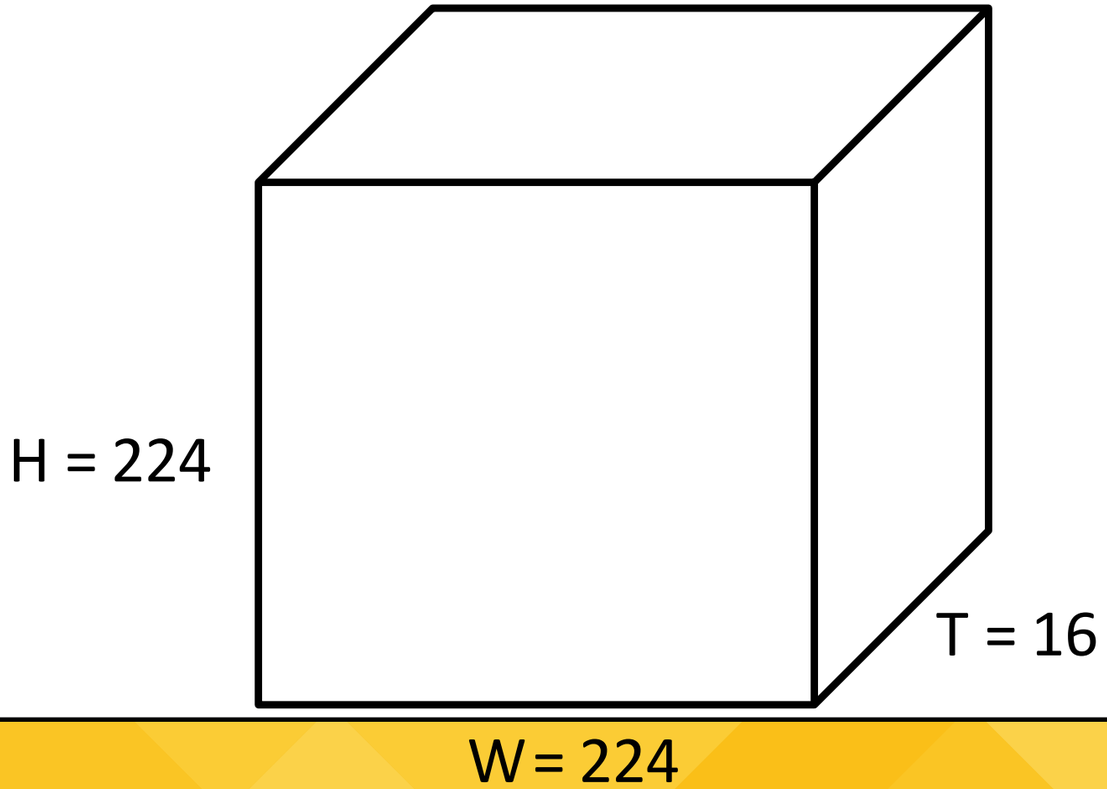
2D Conv (Early Fusion) vs 3D Conv (3D CNN)

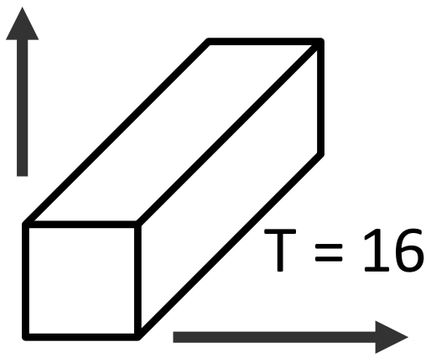
Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

Weight:
C_{out} x C_{in} x T x 3 x 3
Slide over x and y

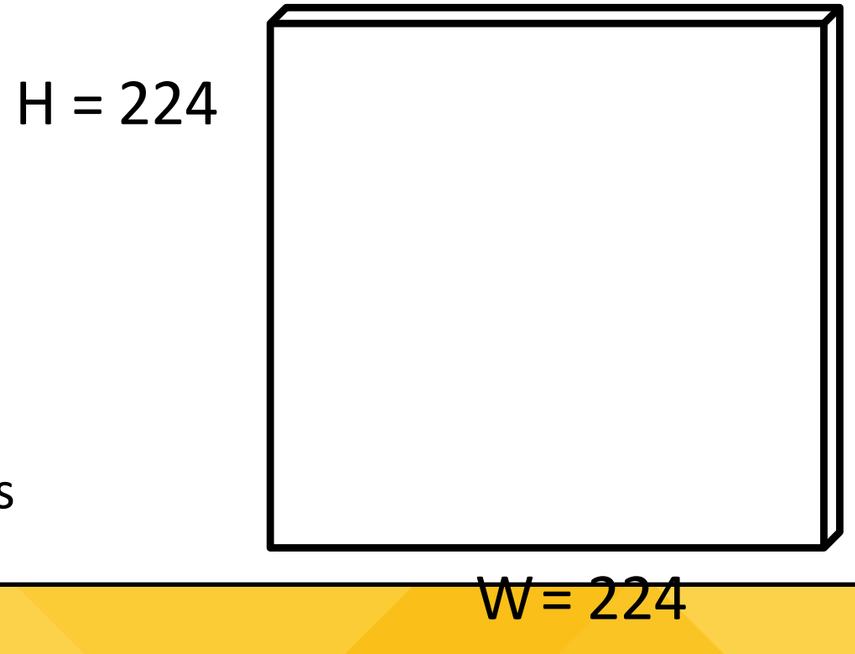
Output:
C_{out} x H x W
2D grid with C_{out} – dim feat at each point

No temporal shift-invariance! Needs to learn separate filters for the same motion at different times in the clip





C_{out} different filters



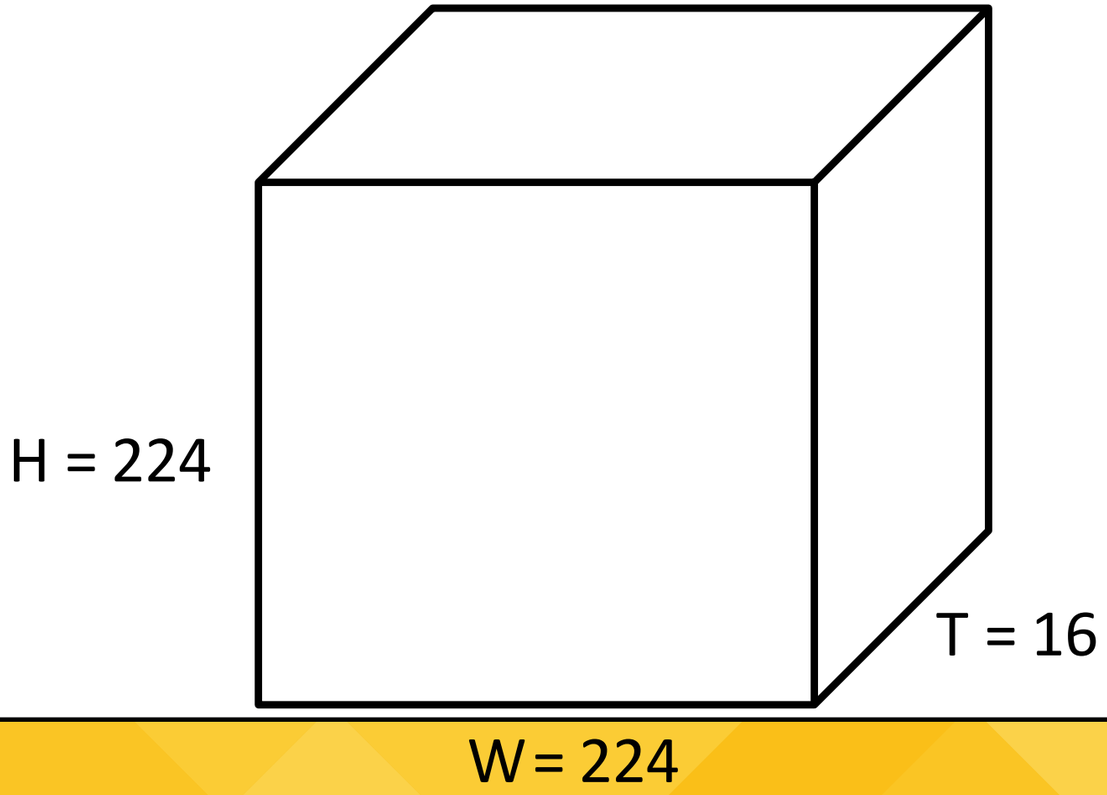
2D Conv (Early Fusion) vs 3D Conv (3D CNN)

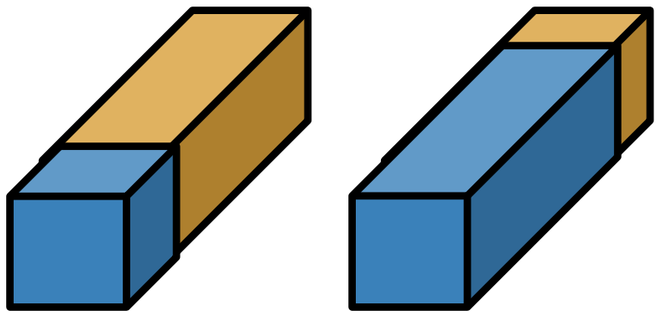
Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

Weight:
C_{out} x C_{in} x T x 3 x 3
Slide over x and y

Output:
C_{out} x H x W
2D grid with C_{out} – dim feat at each point

No temporal shift-invariance! Needs to learn separate filters for the same motion at different times in the clip





C_{out} different filters

How to recognize blue to orange transitions anywhere in space and time?

W = 224

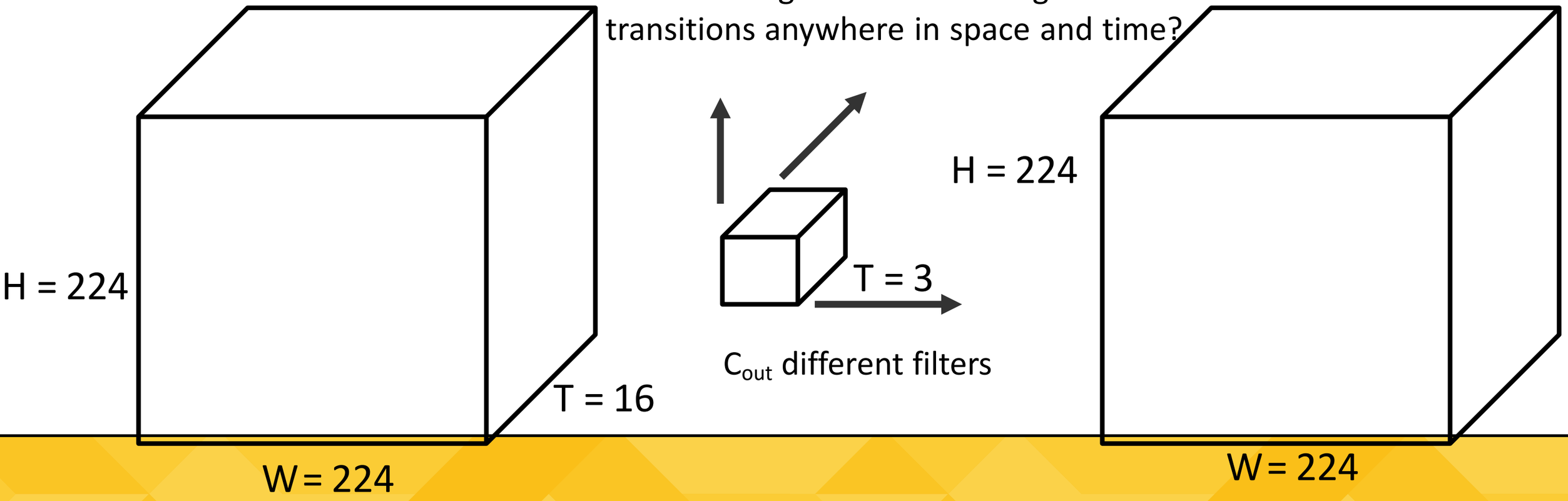
2D Conv (Early Fusion) vs 3D Conv (3D CNN)

Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

Weight:
C_{out} x C_{in} x 3 x 3 x 3
Slide over x and y

Output:
C_{out} x T x H x W
3D grid with C_{out}–dim feat at each point

How to recognize blue to orange transitions anywhere in space and time?



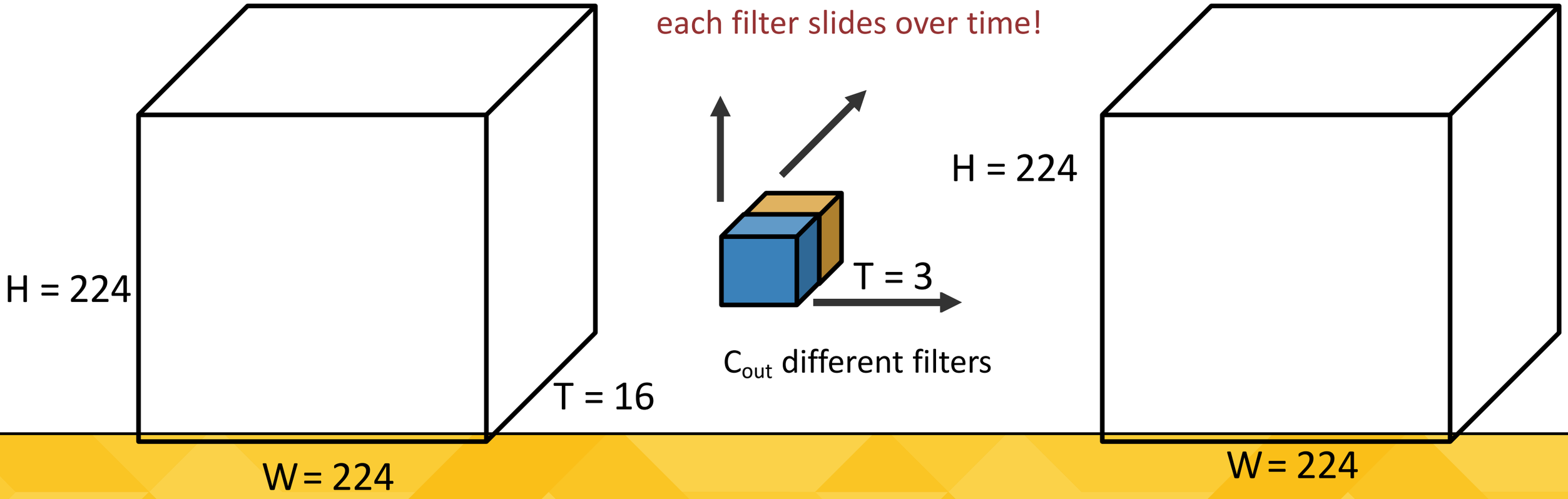
2D Conv (Early Fusion) vs <u>3D Conv (3D CNN)</u>

Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

Weight:
C_{out} x C_{in} x 3 x 3 x 3
Slide over x and y

Output:
C_{out} x T x H x W
3D grid with C_{out}–dim feat at each point

Temporal shift-invariant since each filter slides over time!



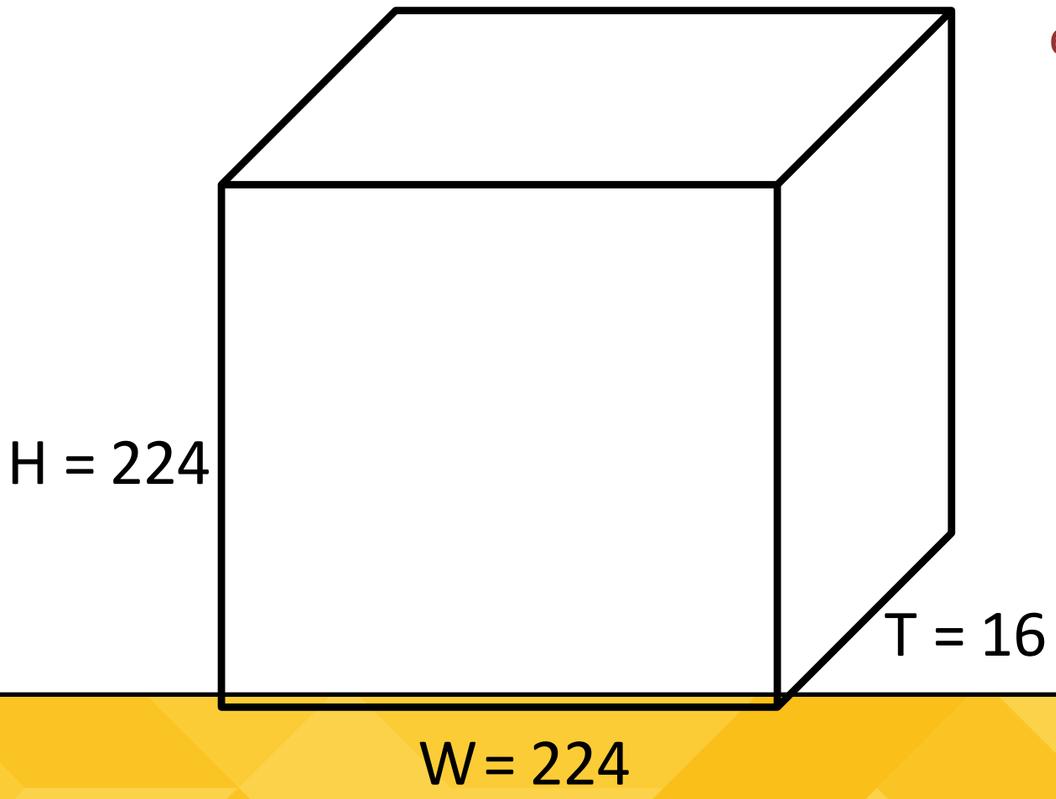
2D Conv (Early Fusion) vs <u>3D Conv (3D CNN)</u>

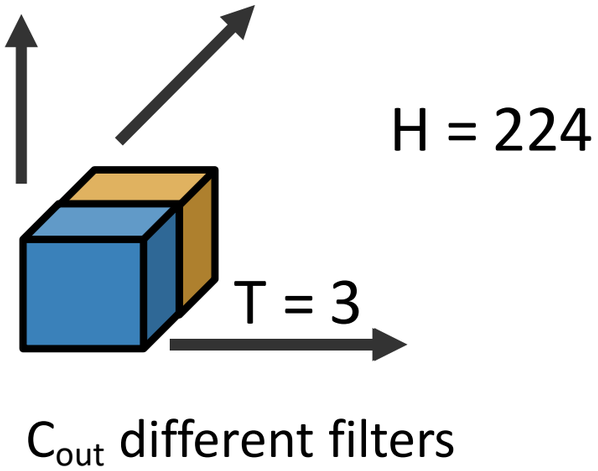
Input: C_{in} x T x H x W
(3D grid with C_{in}-dim feat at each point)

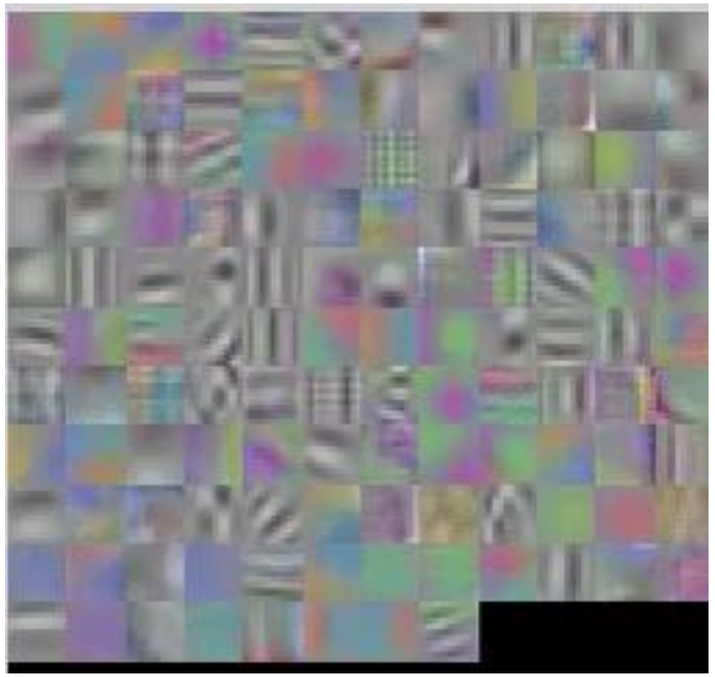
Weight:
C_{out} x C_{in} x 3 x 3 x 3
Slide over x and y

First-layer filters have shape 3 (RGB) x 4 (frames) x 5 x 5 (space)
Can visualize as video clips!

Temporal shift-invariant since each filter slides over time!







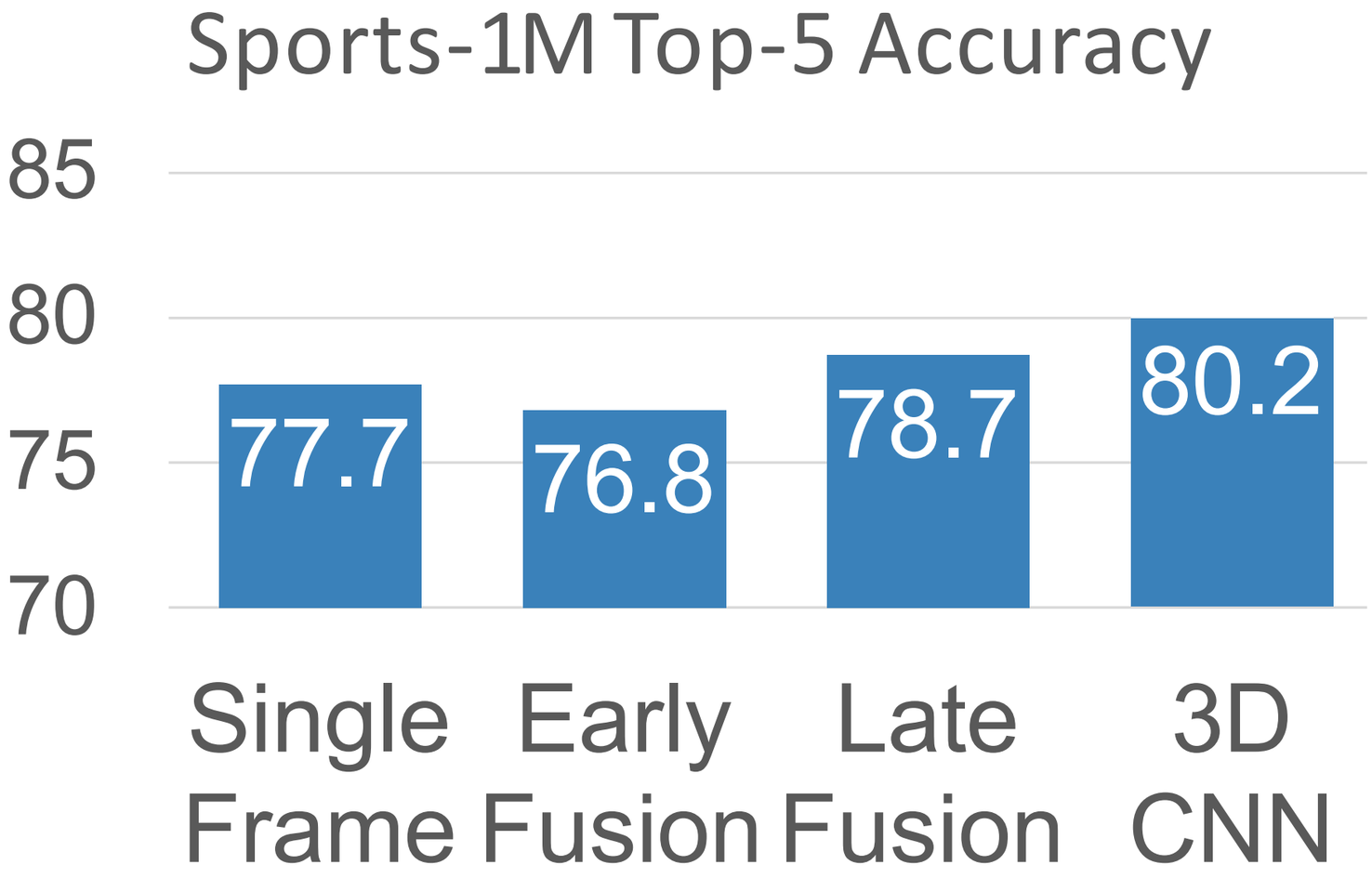
Example Video Dataset: Sports-1M

1 million YouTube videos annotated with labels for 487 different types of sports

Ground Truth
Correct prediction
Incorrect prediction

Karpathy et al, “Large-scale Video Classification with Convolutional Neural Networks”, CVPR 2014

Early Fusion vs Late Fusion vs 3D CNN



- Single Frame model works well – always try this first!
- 3D CNNs have improved a lot since 2014!

Karpathy et al, "Large-scale Video Classification with Convolutional Neural Networks", CVPR 2014

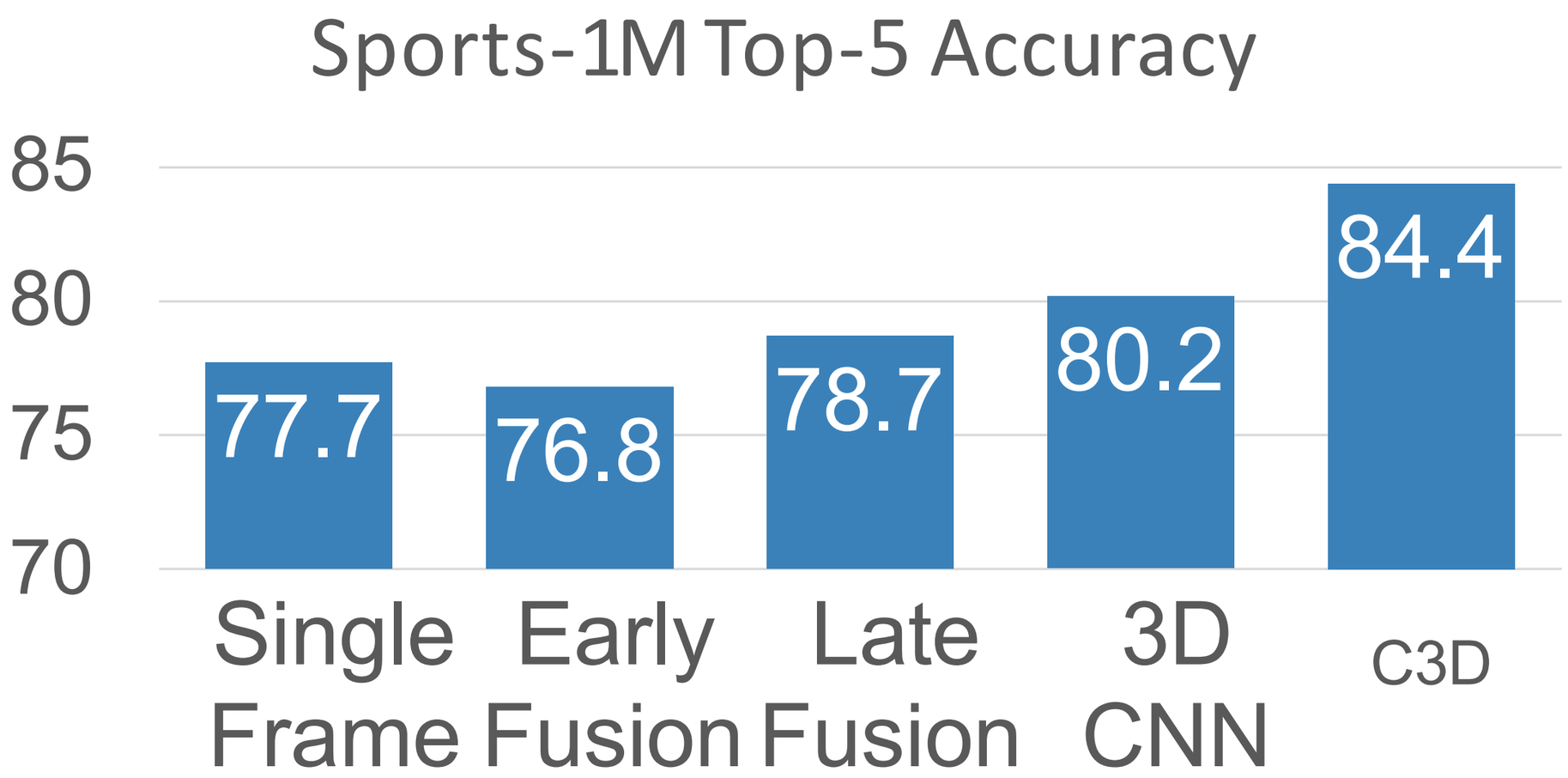
C3D: The VGG of 3D CNNs

- 3D CNN that uses all 3x3x3 conv and 2x2x2 pooling (except Pool1 which is 1x2x2)
- Released model pretrained on Sports-1M: Many people used this as a video feature extractor
- Problem: 3x3x3 conv is very expensive!
- AlexNet: 0.7 GFLOP
- VGG-16: 13.6 GFLOP
- C3D: 39.5 GFLOP (2.9x VGG!)

Layer	Size
Input	3 x 16 x 112 x 112
Conv1 (3x3x3)	64 x 16 x 112 x 112
Pool1 (1x2x2)	64 x 16 x 56 x 56
Conv2 (3x3x3)	128 x 16 x 56 x 56
Pool2 (2x2x2)	128 x 8 x 28 x 28
Conv3a (3x3x3)	256 x 8 x 28 x 28
Conv3b (3x3x3)	256 x 8 x 28 x 28
Pool3 (2x2x2)	256 x 4 x 14 x 14
Conv4a (3x3x3)	512 x 4 x 14 x 14
Conv4b (3x3x3)	512 x 4 x 14 x 14
Pool4 (2x2x2)	512 x 2 x 7 x 7
Conv5a (3x3x3)	512 x 2 x 7 x 7
Conv5b (3x3x3)	512 x 2 x 7 x 7
Pool5	512 x 1 x 3 x 3
FC6	4096
FC7	4096
FC8	C

Tran et al, “Learning Spatiotemporal Features with 3D Convolutional Networks”, ICCV 2015

Early Fusion vs Late Fusion vs 3D CNN



Optical Flow

Recognizing Actions from Motion

- We can easily recognize actions using only motion information

Image at frame t

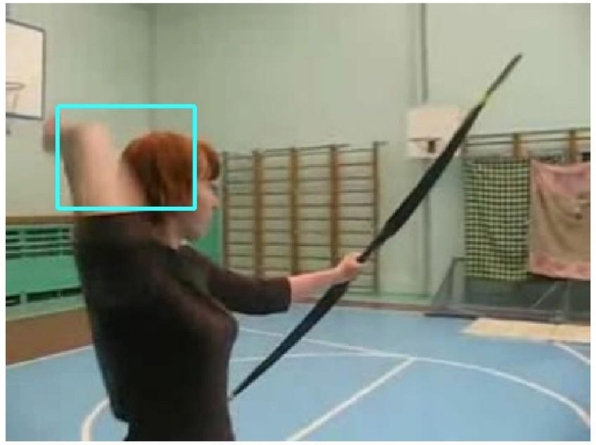
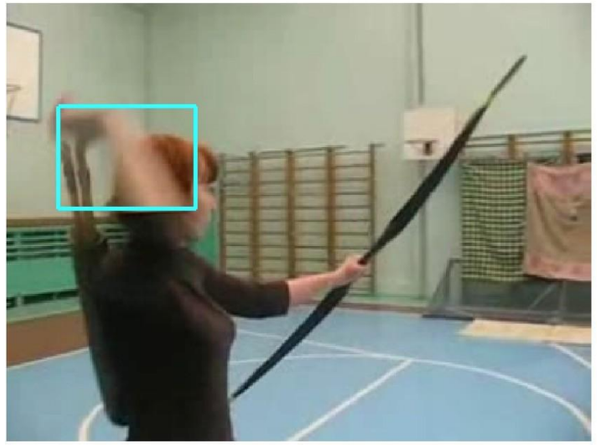


Image at frame t+1



Simonyan and Zisserman, “Two-stream convolutional networks for action recognition in videos”, NeurIPS 2014

Optical Flow

Ideally, Optical Flow = Motion Field

- Motion of brightness patterns in the image

Velocity of brightness pattern

Frame1

Frame2

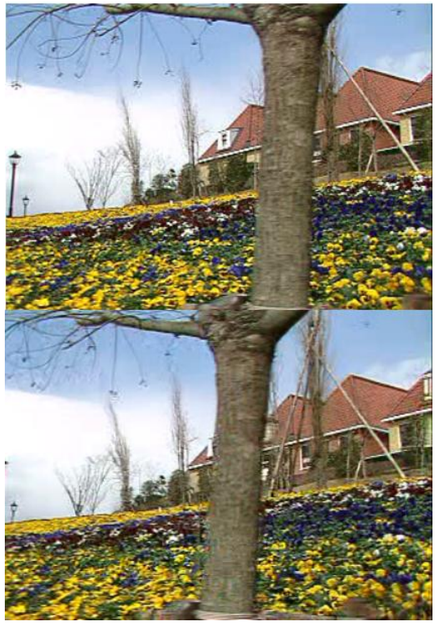
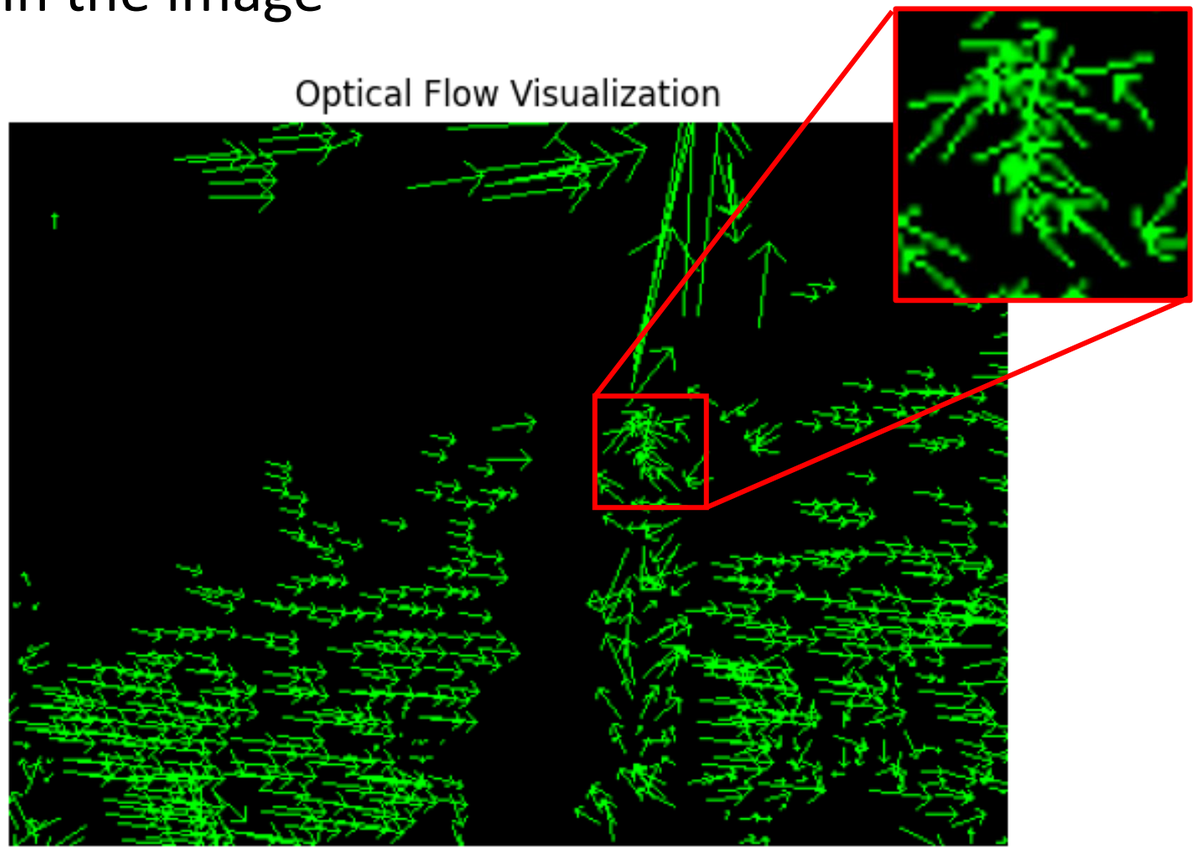
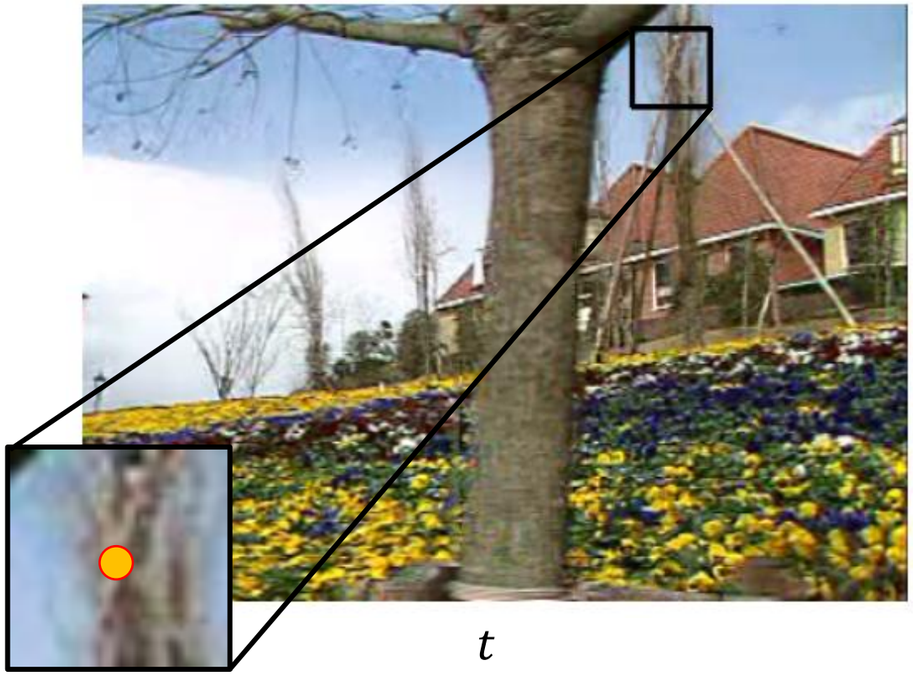


Image Sequence (2 frames)



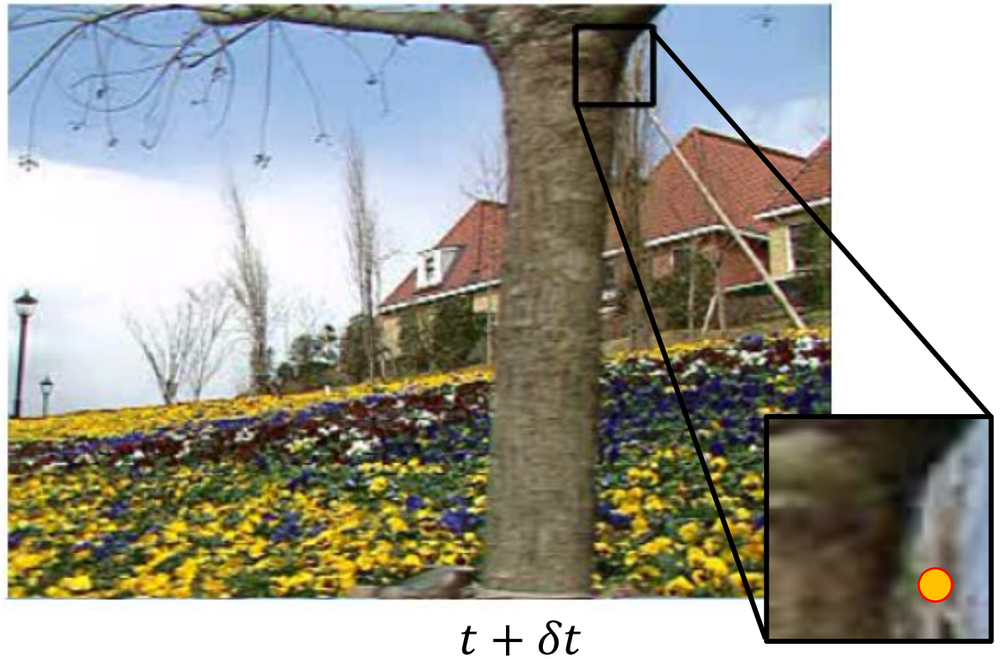
https://ics.uci.edu/~sudderth/papers/uai14hddcrp.pdf

Optical Flow



t

(x, y)



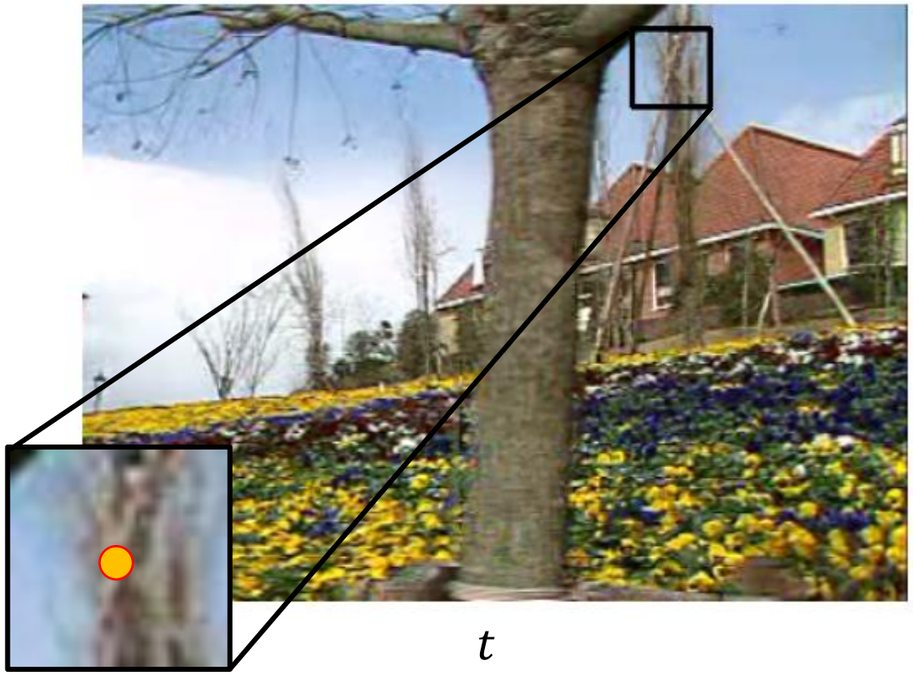
$t + \delta t$

$(x + \delta x, y + \delta y)$

Displacement: $(\delta x, \delta y)$

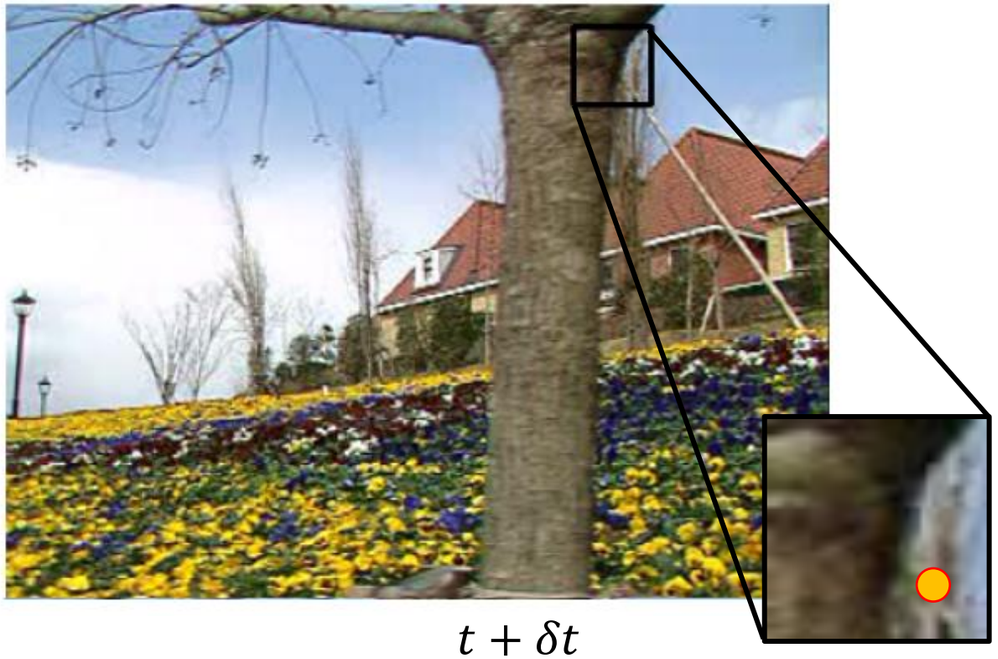
Optical Flow: $(u, v) = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t}\right)$

Optical Flow



t

$I(x, y, t)$



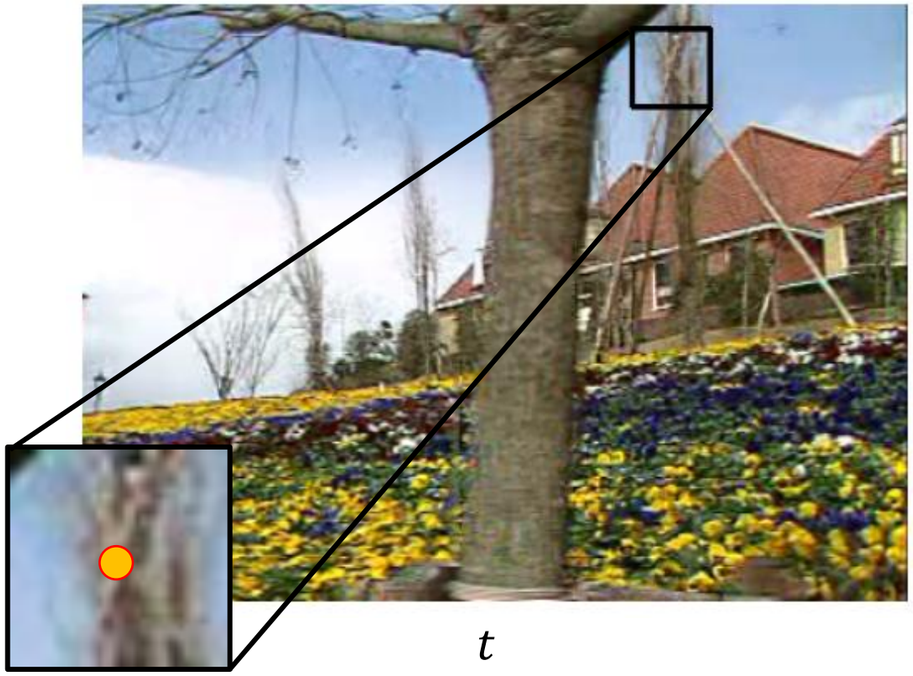
$t + \delta t$

$I(x + \delta x, y + \delta y, t + \delta t)$

Displacement: $(\delta x, \delta y)$

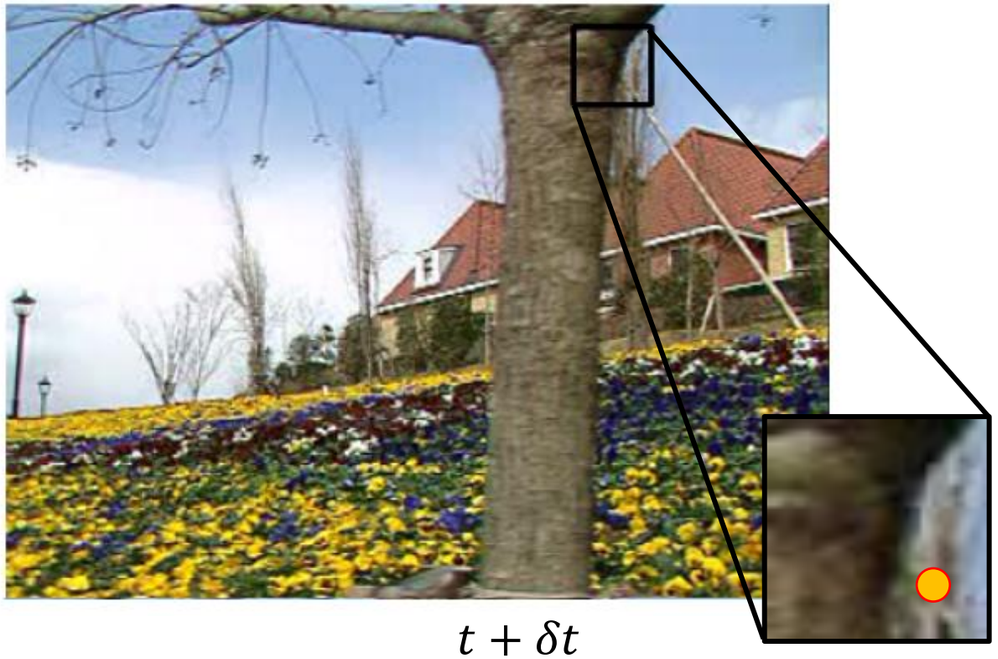
Optical Flow: $(u, v) = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t}\right)$

Optical Flow



t

$I(x, y, t)$



$t + \delta t$

$I(x + \delta x, y + \delta y, t + \delta t)$

Displacement: $(\delta x, \delta y)$

Optical Flow: $(u, v) = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t}\right)$

Assumption #1:
Brightness of image point remains constant over time

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t)$$

Optical Flow Constraint Equation

t

$t + \delta t$

$I(x, y, t)$

$I(x + \delta x, y + \delta y, t + \delta t)$

Displacement: $(\delta x, \delta y)$

Optical Flow: $(u, v) = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t}\right)$

Assumption #2:
Displacement $(\delta x, \delta y)$ and time step δt are small

Taylor Series Expansion

- Expand a function as an infinite sum of its derivatives

$$f(x+\delta x) = f(x) + \frac{\partial f}{\partial x}\delta x + \frac{\partial^2 f}{\partial x^2}\frac{\delta x^2}{2!} + \cdots + \frac{\partial^n f}{\partial x^n}\frac{\delta x^n}{n!}$$

- If δx is small:

$$f(x+\delta x) = f(x) + \frac{\partial f}{\partial x}\delta x + \boxed{O(\delta x^2)} \rightarrow \text{Almost zero}$$

- For a function of three variables with small δx, δy, δt

$$f(x+\delta x, y+\delta y, t+\delta t) = f(x,y,t) + \frac{\partial f}{\partial x}\delta x + \frac{\partial f}{\partial y}\delta y + \frac{\partial f}{\partial t}\delta t + O(\delta x^2) + O(\delta y^2) + O(\delta z^2)$$

$$f(x+\delta x, y+\delta y, t+\delta t) \approx f(x,y,t) + \frac{\partial f}{\partial x}\delta x + \frac{\partial f}{\partial y}\delta y + \frac{\partial f}{\partial t}\delta t$$

Optical Flow Constraint Equation

- Assumption #2:
- Displacement $(\delta x, \delta y)$ and time step δt are small
- Thus

$$I(x+\delta x, y+\delta y, t+\delta t) = I(x,y,t) + \frac{\partial I}{\partial x}\delta x + \frac{\partial I}{\partial y}\delta y + \frac{\partial I}{\partial t}\delta t$$

$$I(x+\delta x, y+\delta y, t+\delta t) = I(x,y,t) + I_x\delta x + I_y\delta y + I_t\delta t$$

Optical Flow Constraint Equation

- Assumption #1: Brightness of image point remains constant over time

$$I(x+\delta x, y+\delta y, t+\delta t) = I(x,y,t) \ (1)$$

- Assumption #2: Displacement $(\delta x, \delta y)$ and time step δt are small

$$I(x+\delta x, y+\delta y, t+\delta t) = I(x,y,t) + I_x \delta x + I_y \delta y + I_t \delta t \ (2)$$

- Subtract (1) from (2):

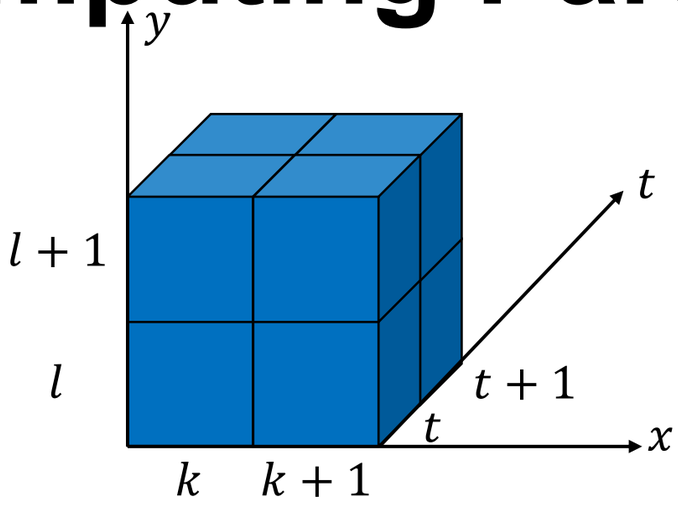
$$I_x \delta x + I_y \delta y + I_t \delta t = 0$$

- Divide by δt and take limit as $\delta t \to 0$:

$$I_x \frac{\partial x}{\partial t} + I_y \frac{\partial y}{\partial t} + I_t = 0$$

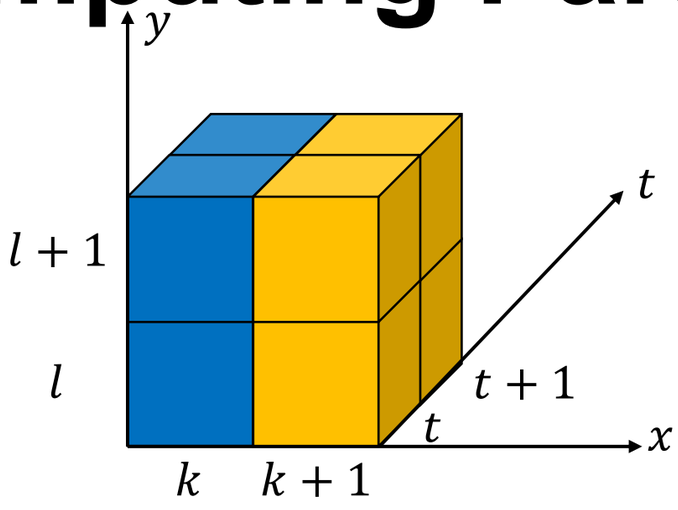
- Constraint Equation: $I_x u + I_y v + I_t = 0$, where $(u, v) = \left(\frac{\delta x}{\delta t}, \frac{\delta y}{\delta t}\right)$ is the optical flow

Computing Partial Derivatives I_x, I_y, I_t



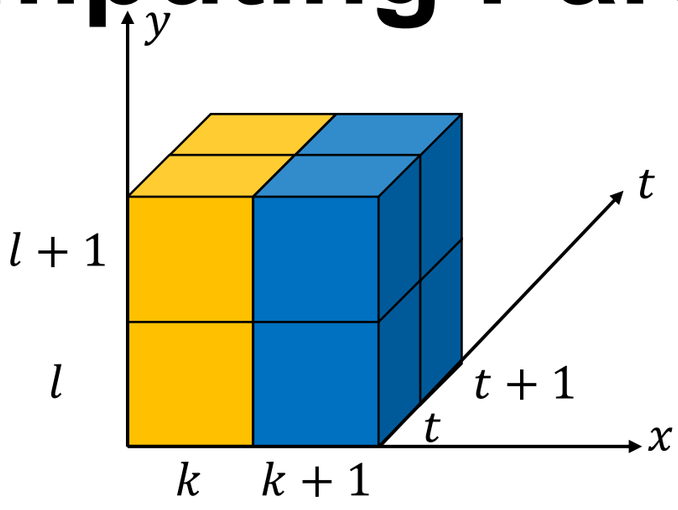
$$I_x(k, l, t) = ?$$

Computing Partial Derivatives I_x, I_y, I_t



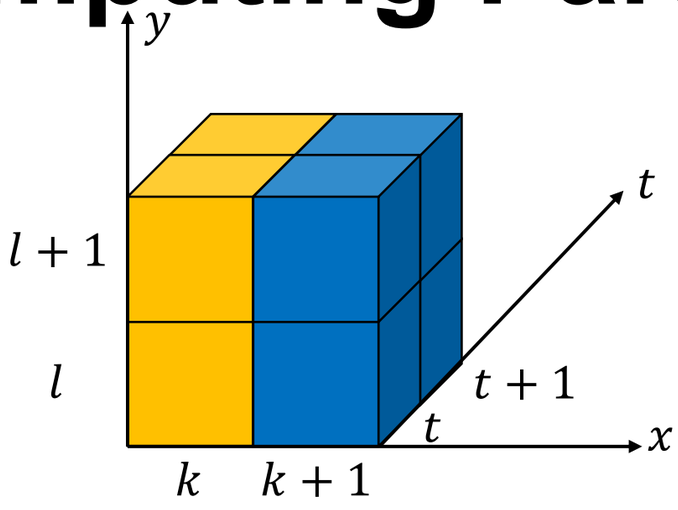
$$I_x(k,l,t)$$
$$= \frac{1}{4}[I(k+1,l,t) + I(k+1,l,t+1) + I(k+1,l+1,t) + I(k+1,l+1,t+1)]$$

Computing Partial Derivatives I_x, I_y, I_t



$$
\begin{aligned}
&I_x(k,l,t) \\
&= \frac{1}{4}\left[I(k+1,l,t) + I(k+1,l,t+1) + I(k+1,l+1,t) + I(k+1,l+1,t+1)\right] \\
&\qquad - \frac{1}{4}\left[I(k,l,t) + I(k,l,t+1) + I(k,l+1,t) + I(k,l+1,t+1)\right]
\end{aligned}
$$

Computing Partial Derivatives I_x, I_y, I_t



$$
\begin{aligned}
&I_x(k,l,t) \\
&= \frac{1}{4}[I(k+1,l,t) + I(k+1,l,t+1) + I(k+1,l+1,t) + I(k+1,l+1,t+1)] \\
&\qquad - \frac{1}{4}[I(k,l,t) + I(k,l,t+1) + I(k,l+1,t) + I(k,l+1,t+1)]
\end{aligned}
$$

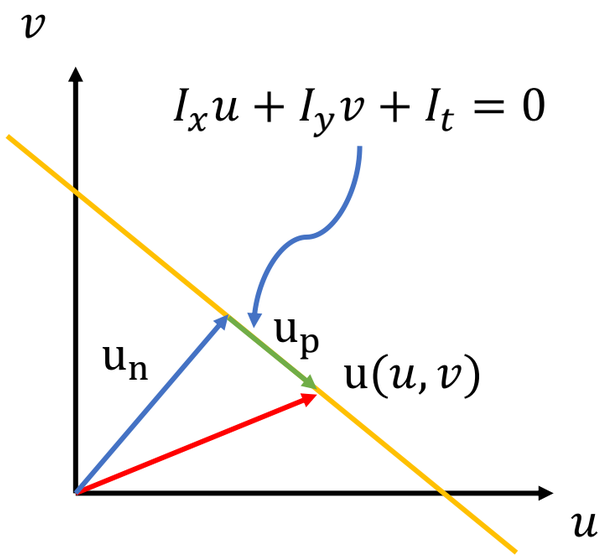
Similarly find $I_y(k,l,t)$ and $I_t(k,l,t)$

Geometric Interpretation

- For any point (x, y) in the image, its optical flow (u, v) lies on the line:

$$I_x u + I_y v + I_t = 0$$

- Optical flow can be split into two components
- $\mathrm{u} = \mathrm{u_n} + \mathrm{u_p}$
- where $\mathrm{u_n}$: normal flow, $\mathrm{u_p}$ parallel flow



Normal Flow

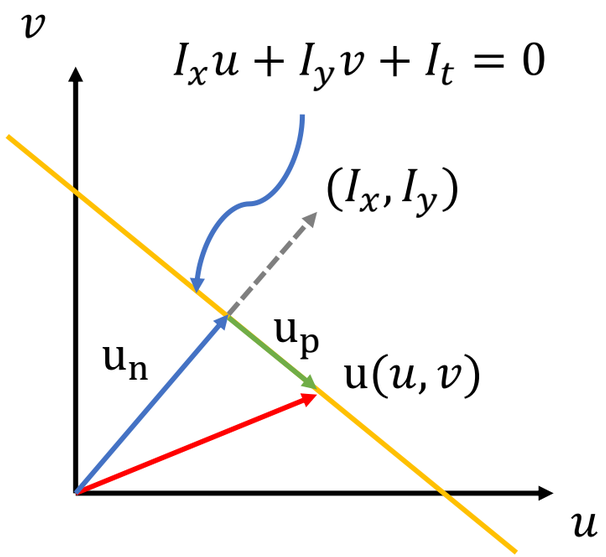
- Direction of Normal Flow:
- Unit vector perpendicular to the constraint line

$$\hat{u}_n = \frac{(I_x, I_y)}{\sqrt{I_x^2 + I_y^2}}$$

- Magnitude of Normal Flow:
- Distance of origin from the constraint line

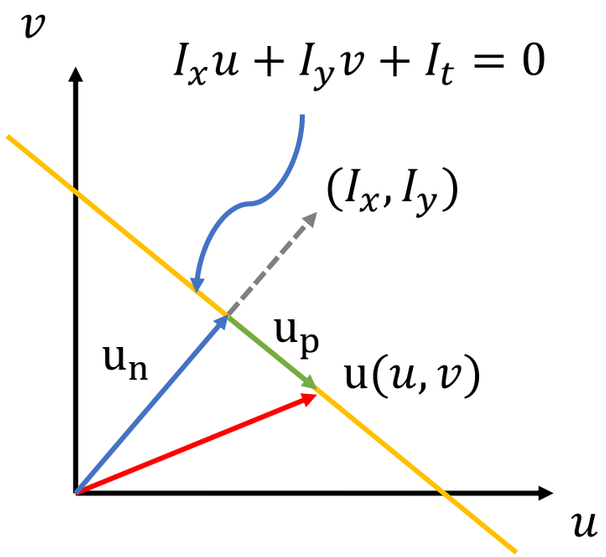
$$|u_n| = \frac{|I_t|}{\sqrt{I_x^2 + I_y^2}}$$

Then, $u_n = \frac{|I_t|}{(I_x^2 + I_y^2)}(I_x, I_y)$



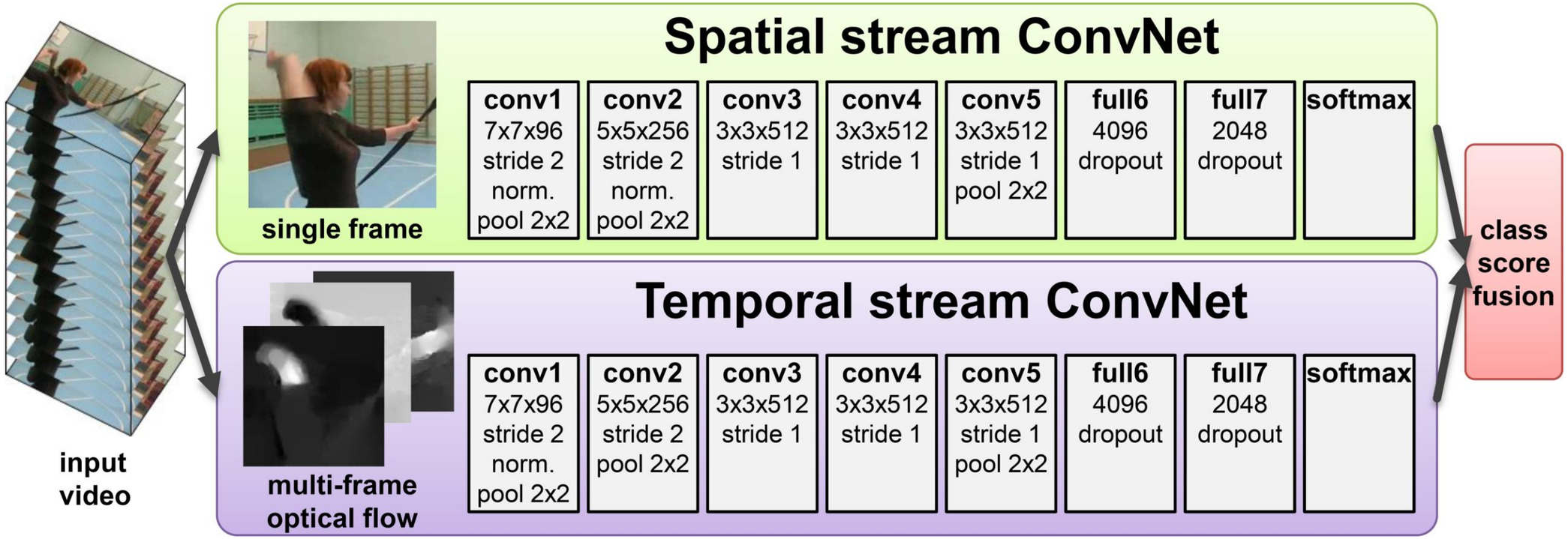
Parallel Flow

- We cannot determine $\mathrm{u_p}$
- The optical flow component parallel to the constraint line



Separating Motion and Appearance: Two-Stream Networks

Input: Single Image
3 x H x W

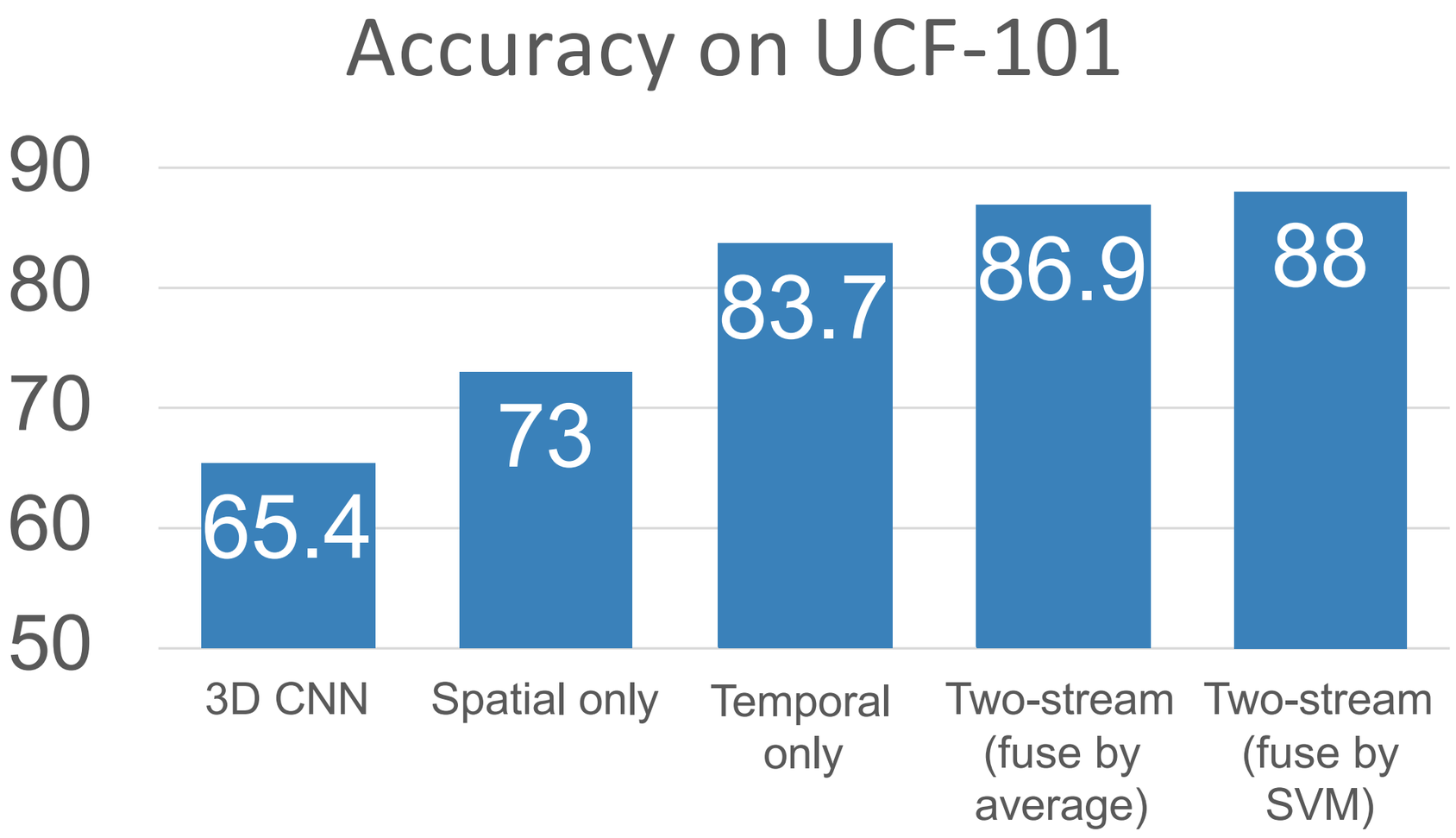


Input: Stack of optical flow:
[2*(T-1)] x H x W

Early fusion: First 2D conv processes all flow images

Simonyan and Zisserman, “Two-stream convolutional networks for action recognition in videos”, NeurIPS 2014

Separating Motion and Appearance: Two-Stream Networks

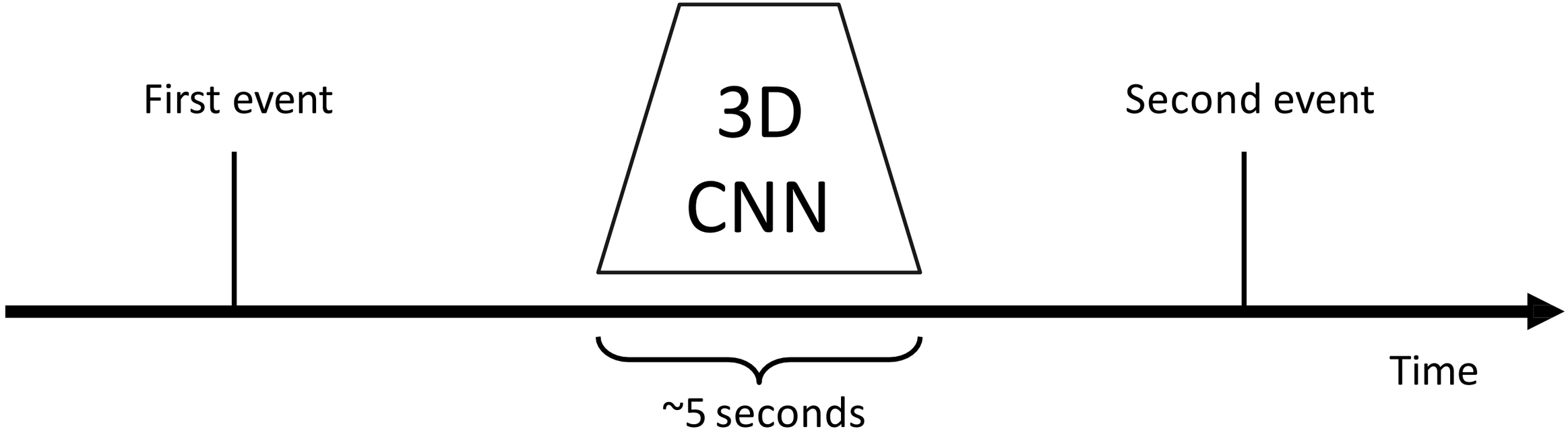


Simonyan and Zisserman, “Two-stream convolutional networks for action recognition in videos”, NeurIPS 2014

RNN for Video Understanding

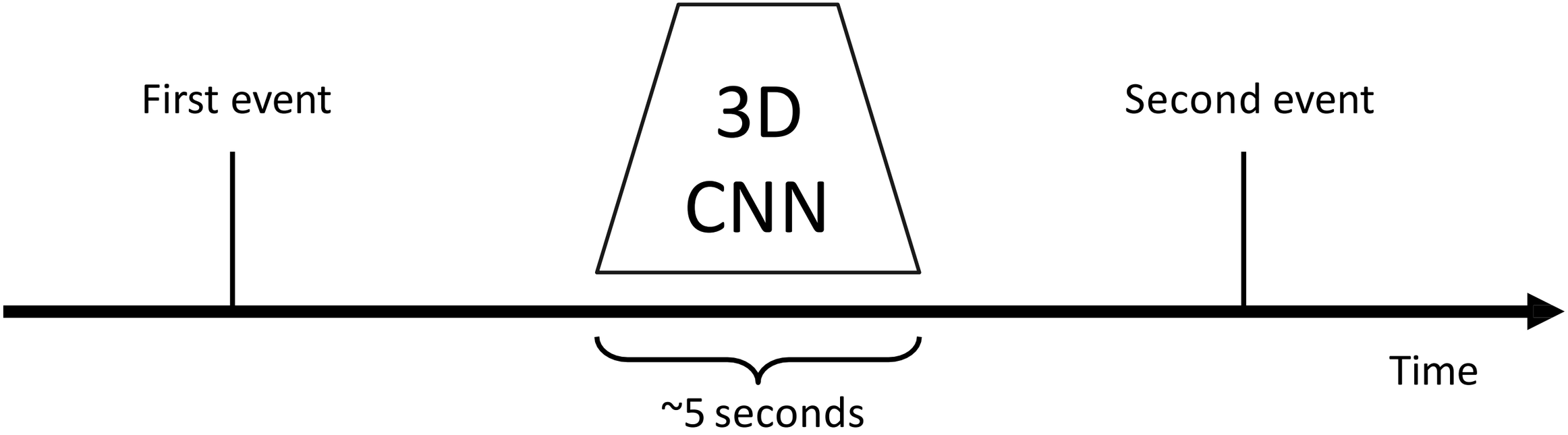
Modeling long-term temporal structure

- So far all our temporal CNNs only model local motion between frames in very short clips of ~2-5 seconds. What about long-term structure?

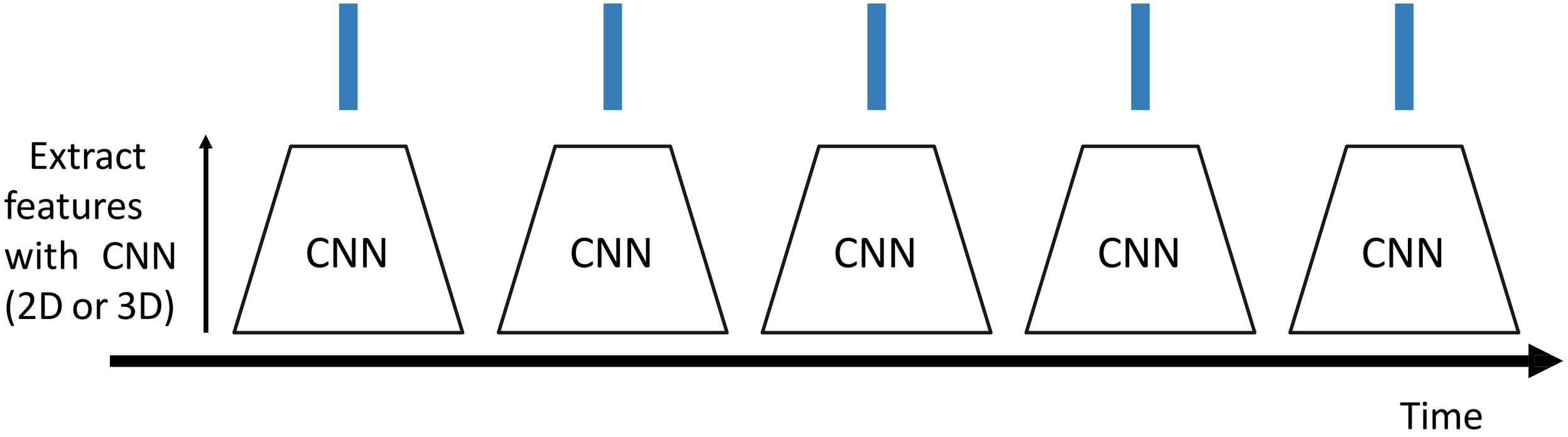


Modeling long-term temporal structure

- So far all our temporal CNNs only model local motion between frames in very short clips of ~2-5 seconds. What about long-term structure?
- We know how to handle sequences! How about recurrent networks?

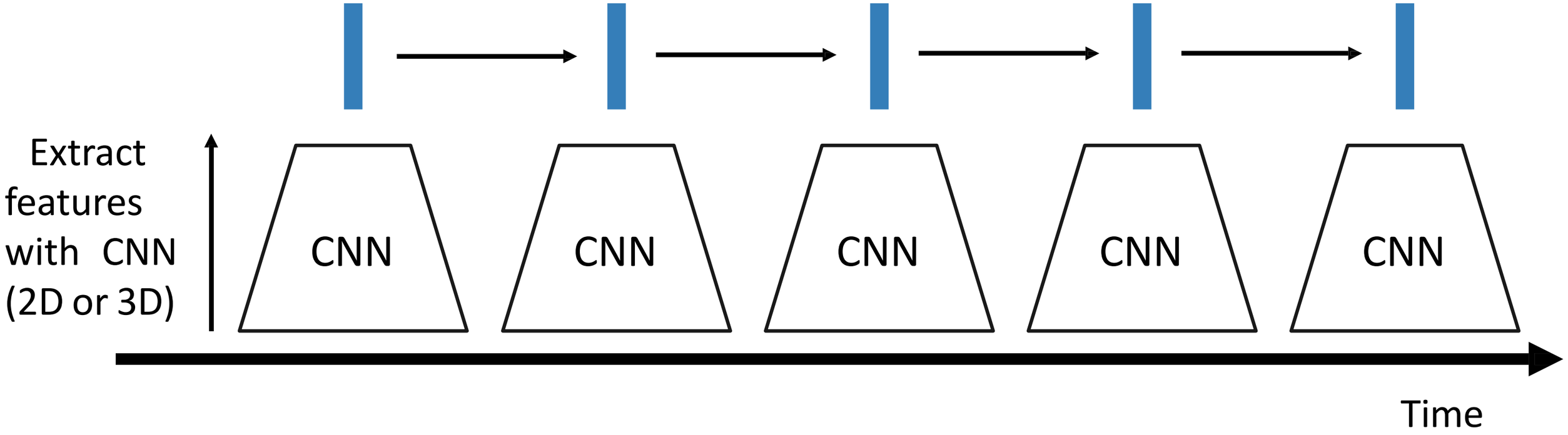


Modeling long-term temporal structure



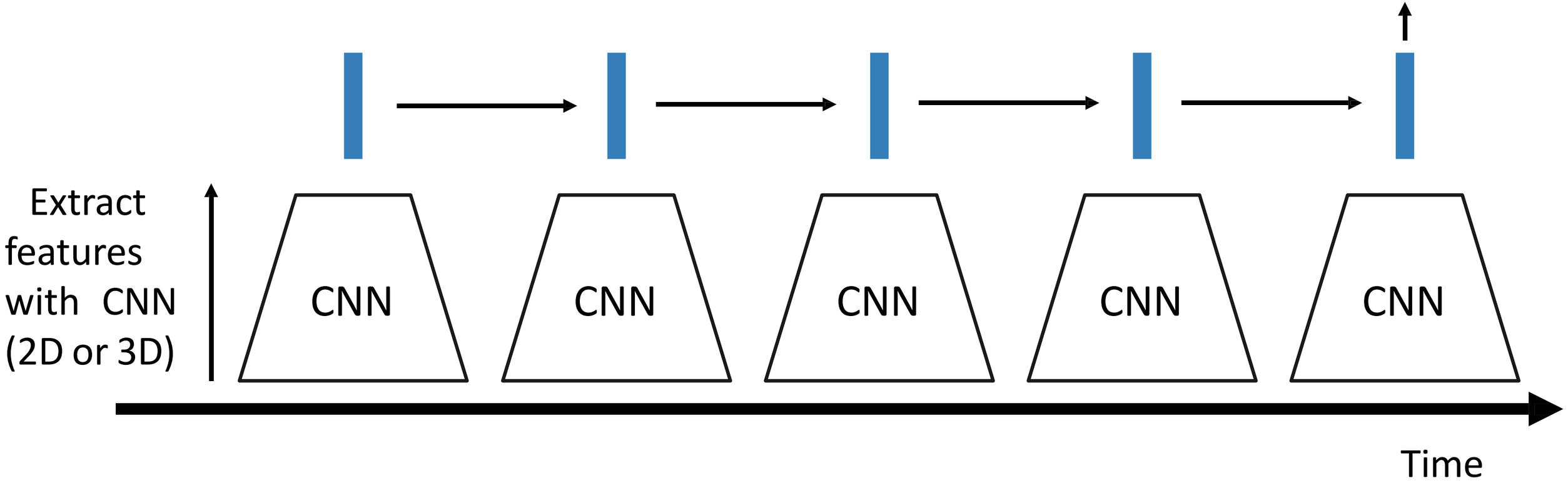
Modeling long-term temporal structure

Process local features using recurrent network (e.g. LSTM)



Modeling long-term temporal structure

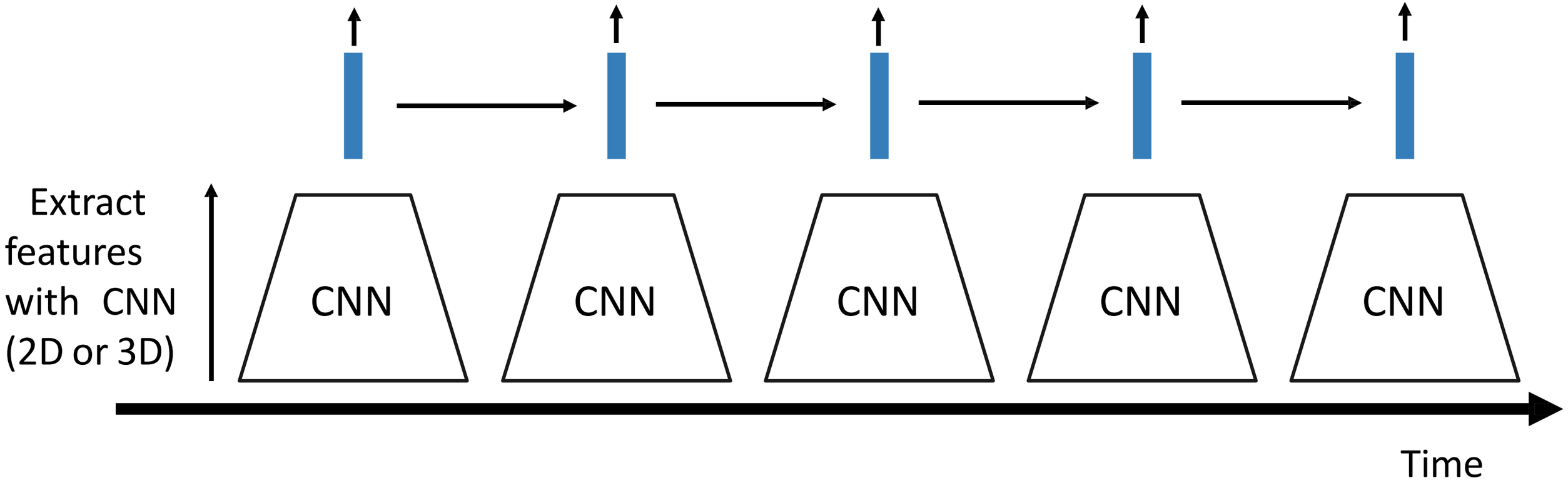
Process local features using recurrent network (e.g. LSTM)
Many to one: One output at end of video



Modeling long-term temporal structure

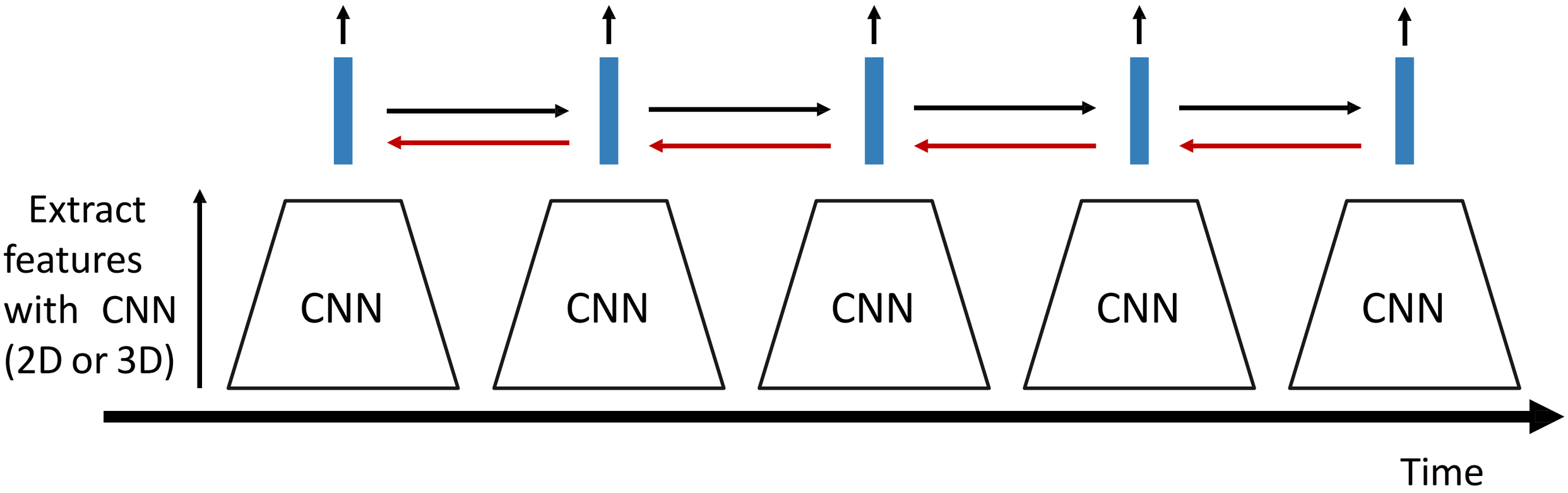
Process local features using recurrent network (e.g. LSTM)

Many to many: one output per video frame



Modeling long-term temporal structure

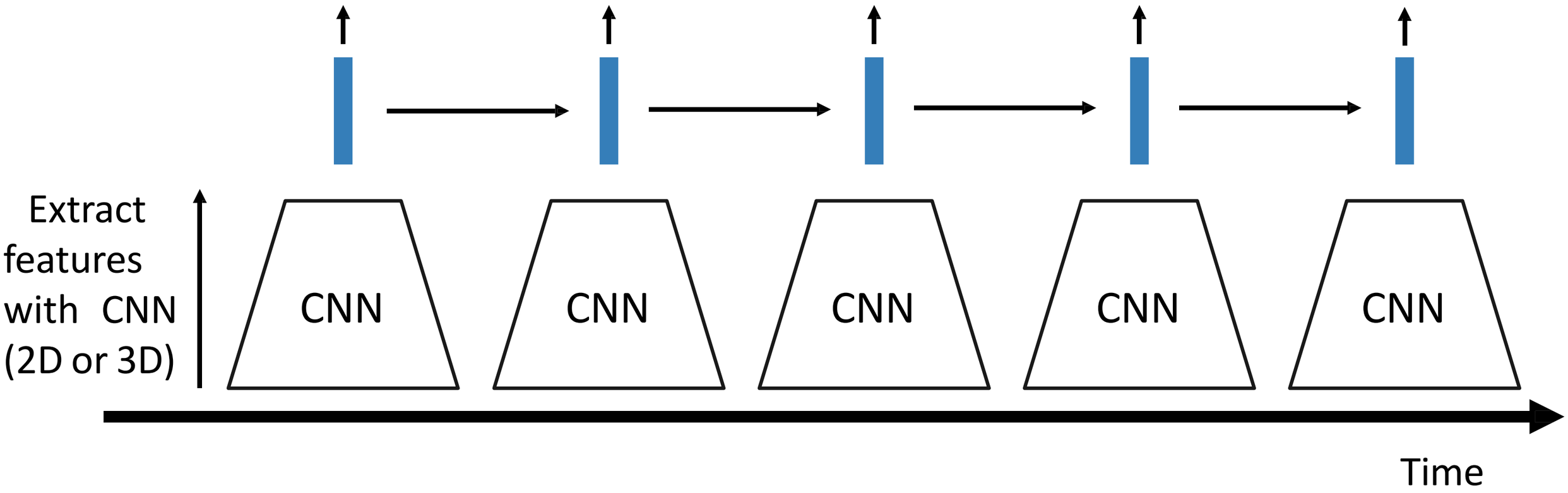
Sometimes don't backprop to CNN to save memory;
pretrain and use it as a feature extractor



Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011
Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Modeling long-term temporal structure

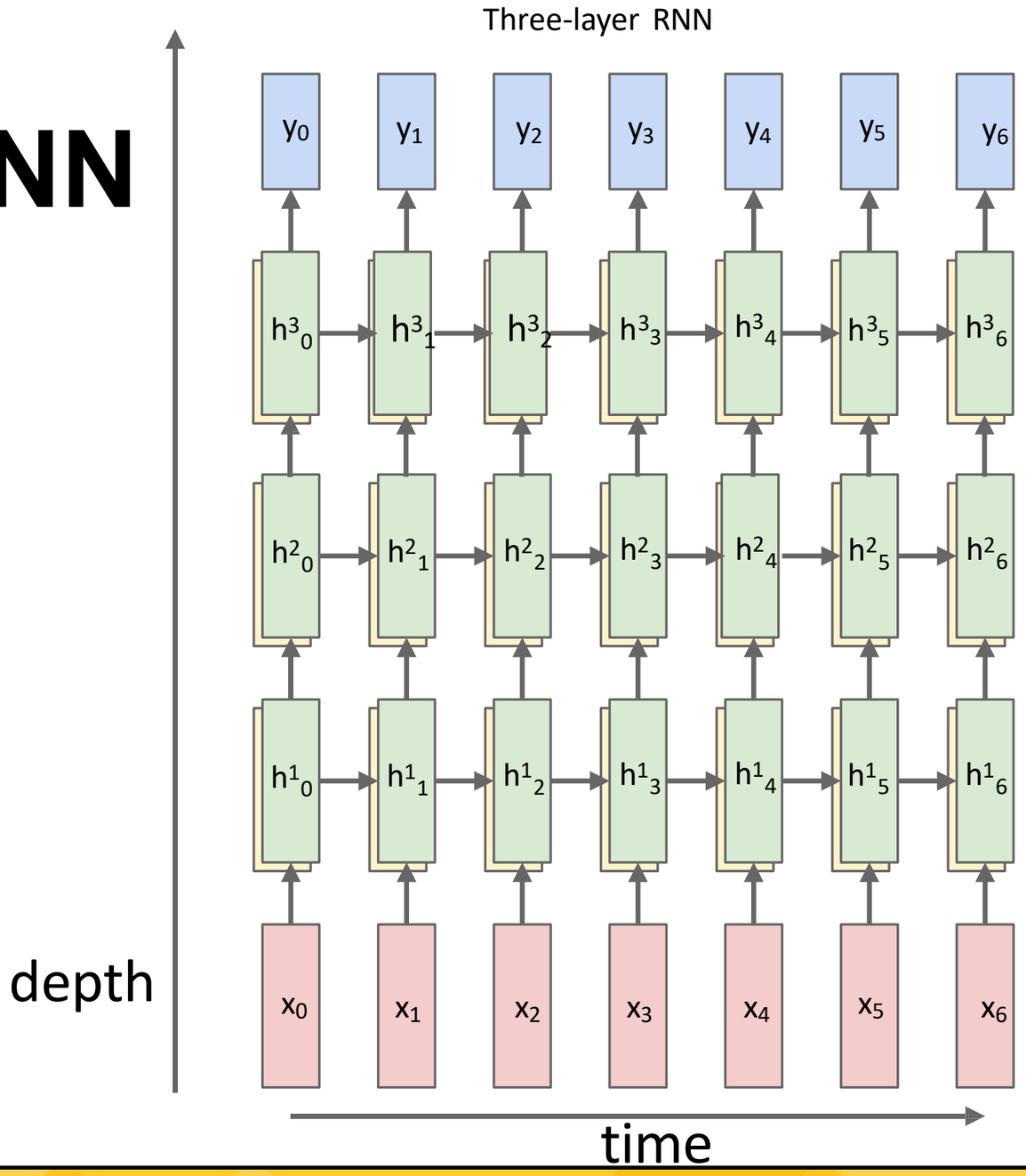
Inside CNN: Each value is a function of a fixed temporal window (local temporal structure) Inside RNN: Each vector is a function of all previous vectors (global temporal structure) Can we merge both approaches?



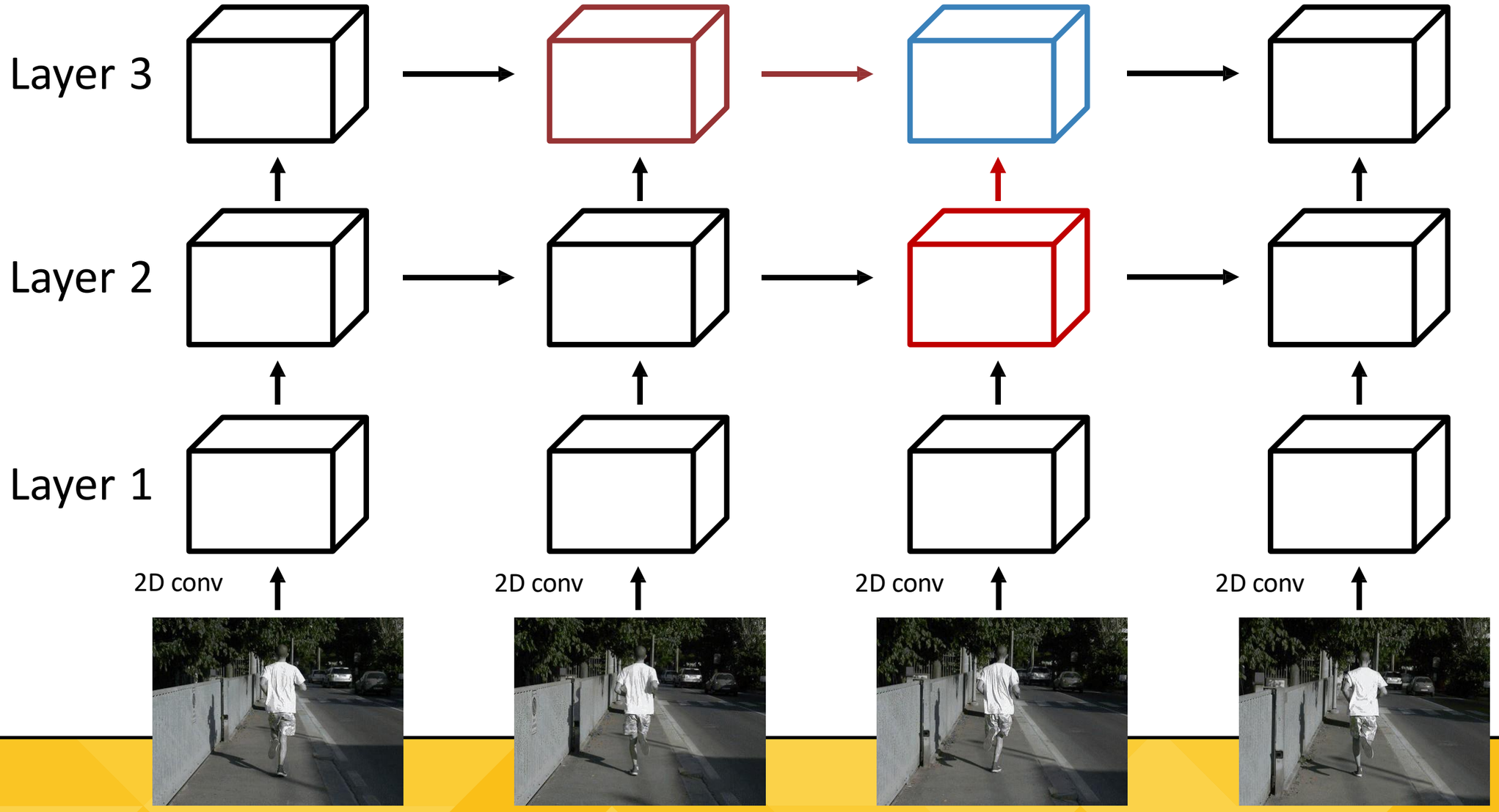
Baccouche et al, "Sequential Deep Learning for Human Action Recognition", 2011
Donahue et al, "Long-term recurrent convolutional networks for visual recognition and description", CVPR 2015

Recall: Multi-layer RNN

- We can use a similar structure to process videos!



Recurrent Convolutional Network



Entire network uses 2D feature maps: C x H x W

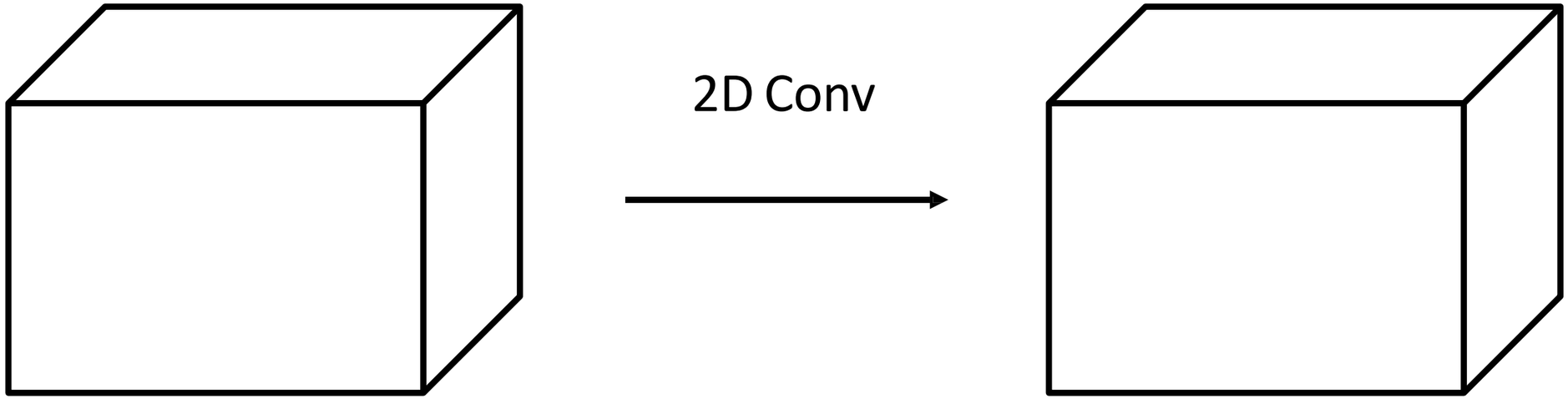
Each depends on two inputs:
1. Same layer, previous timestep
2. Prev layer, same timestep

Use different weights at each layer, share weights across time

Ballas et al, "Delving Deeper into Convolutional Networks for Learning Video Representations", ICLR 2016

Recurrent Convolutional Network

- Normal 2D CNN:



Input features:
C x H x W

Output features:
C x H x W

Recurrent Convolutional Network

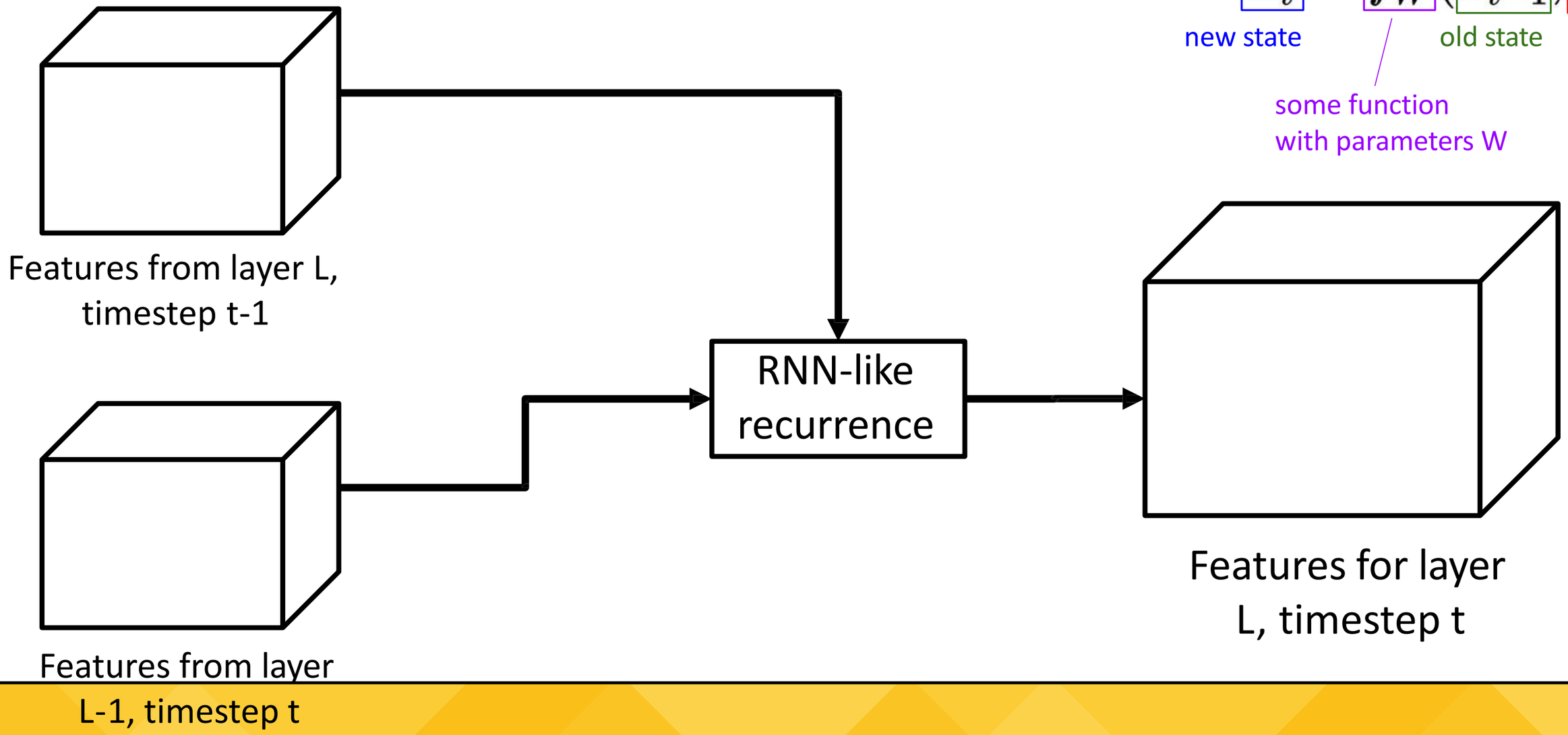
Recall: Recurrent Network

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

old state

some function
with parameters W



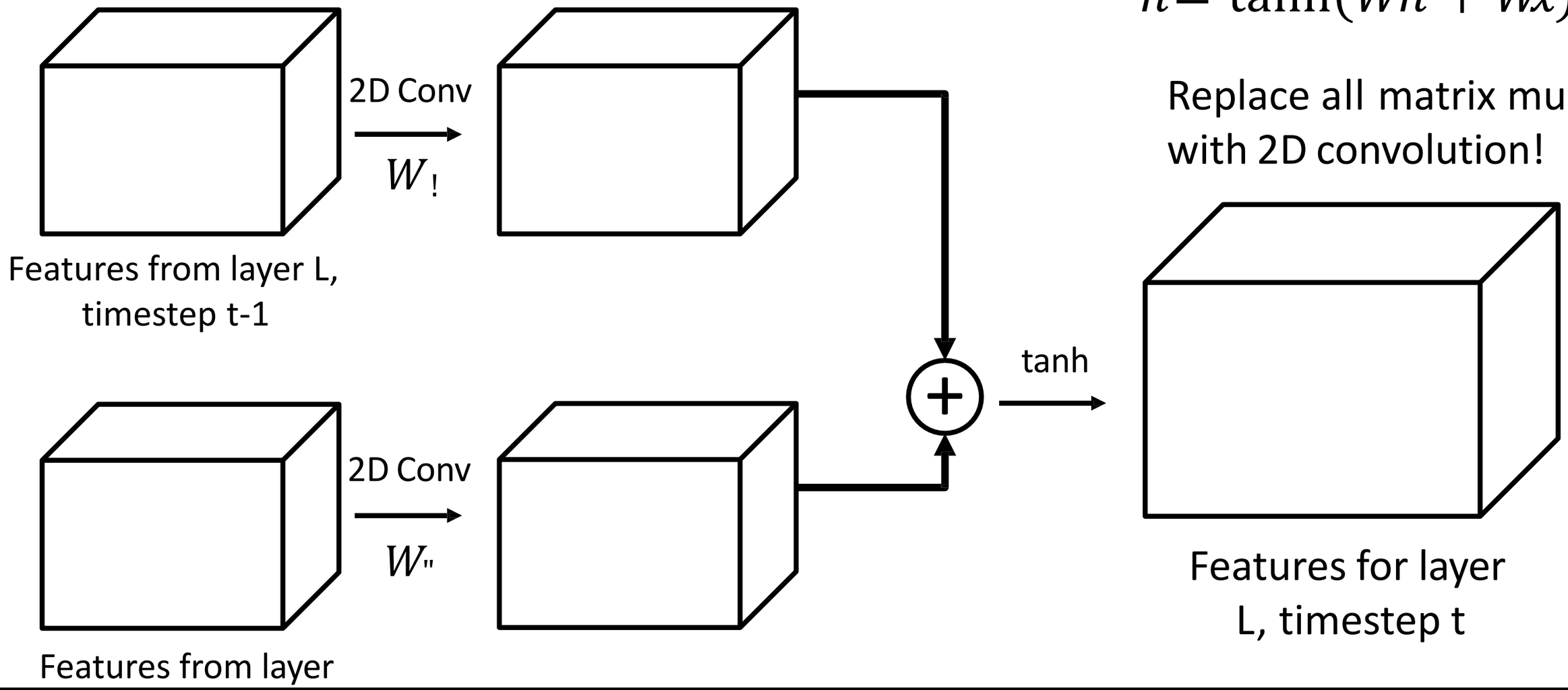
Ballas et al. "Delving Deeper into Convolutional Networks for Learning Video Representations", ICLR 2016

Recurrent Convolutional Network

Recall: Vanilla RNN

$$h = \tanh(Wh + Wx)$$

Replace all matrix multiply with 2D convolution!

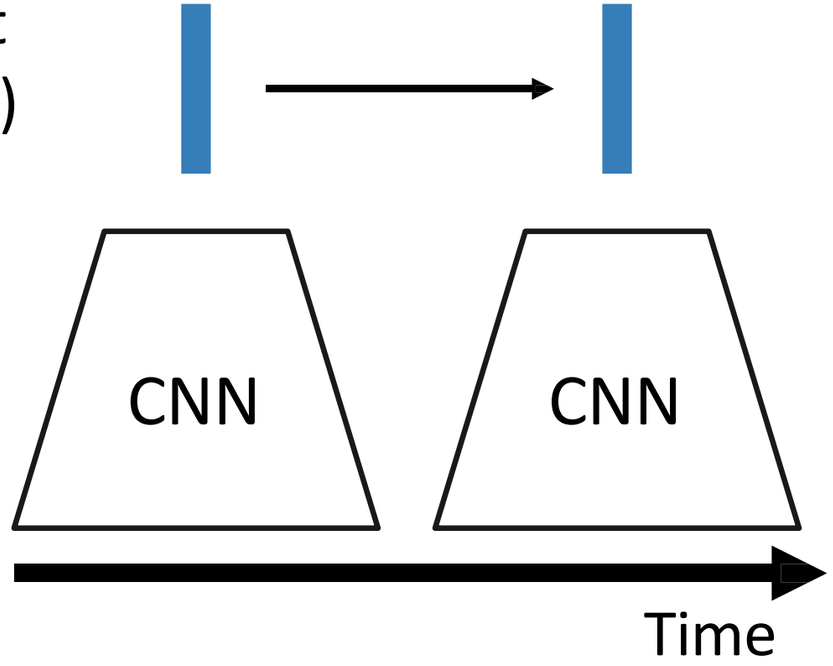


Modeling long-term temporal structure

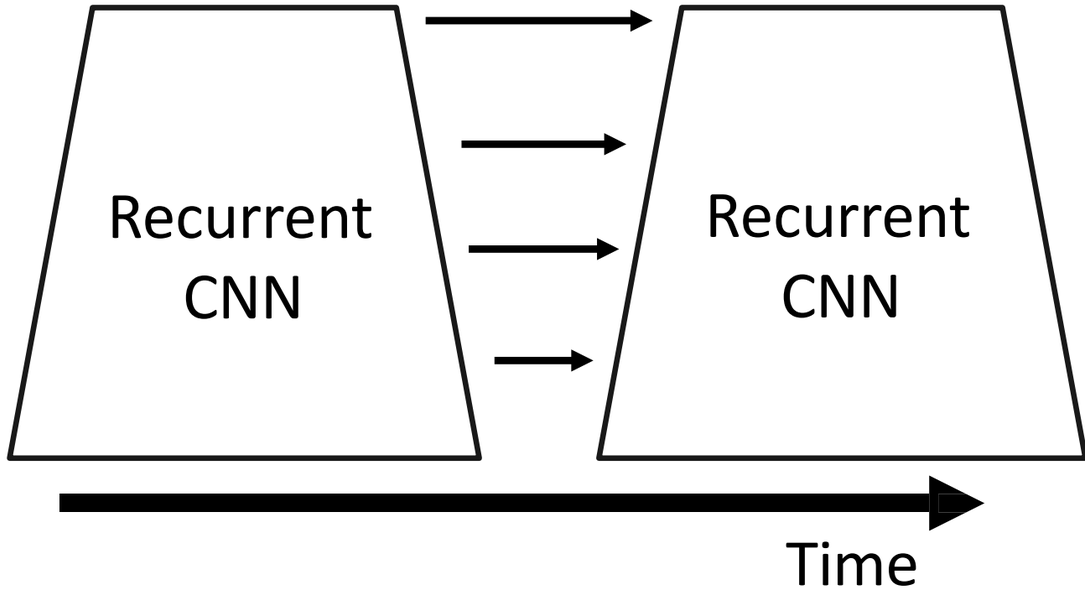
Problem: RNNs are slow for long sequences (can't be parallelized)

RNN: Infinite temporal extent (fully-connected)

CNN: finite temporal extent (convolutional)

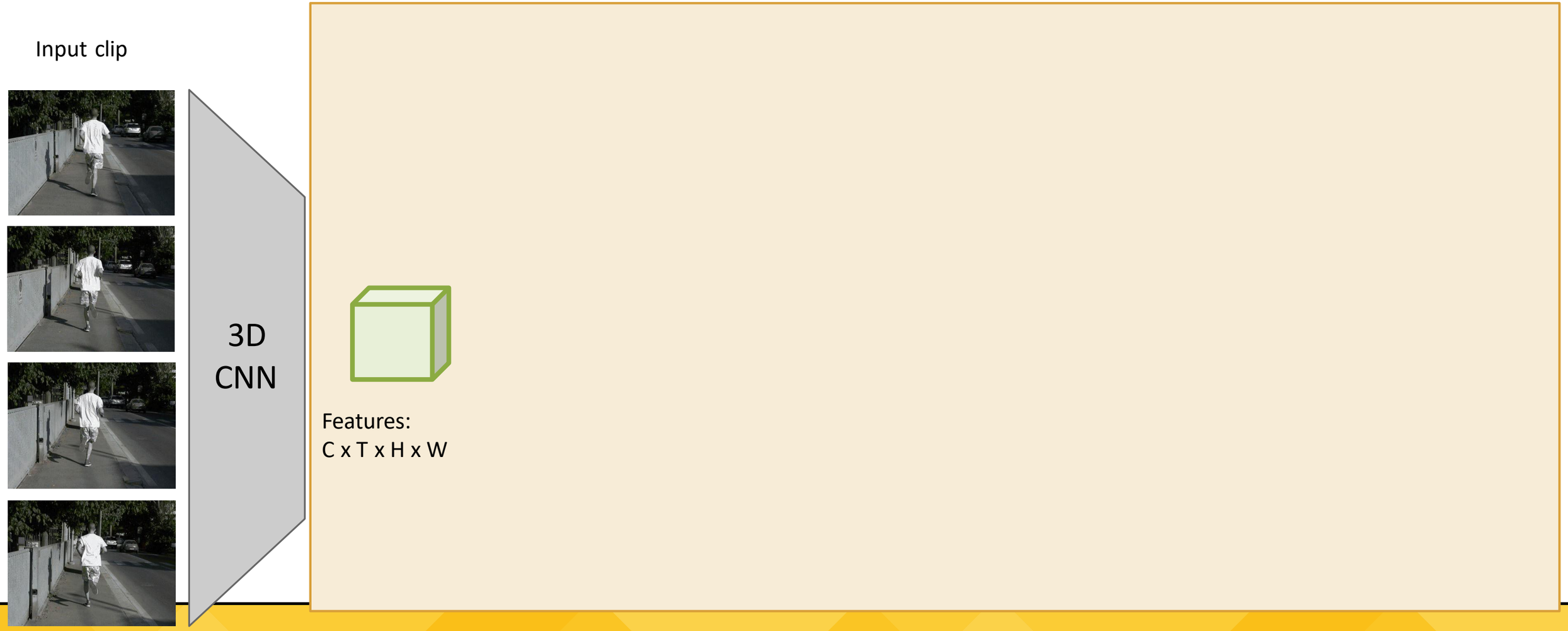


Recurrent CNN: Infinite temporal extent (convolutional)

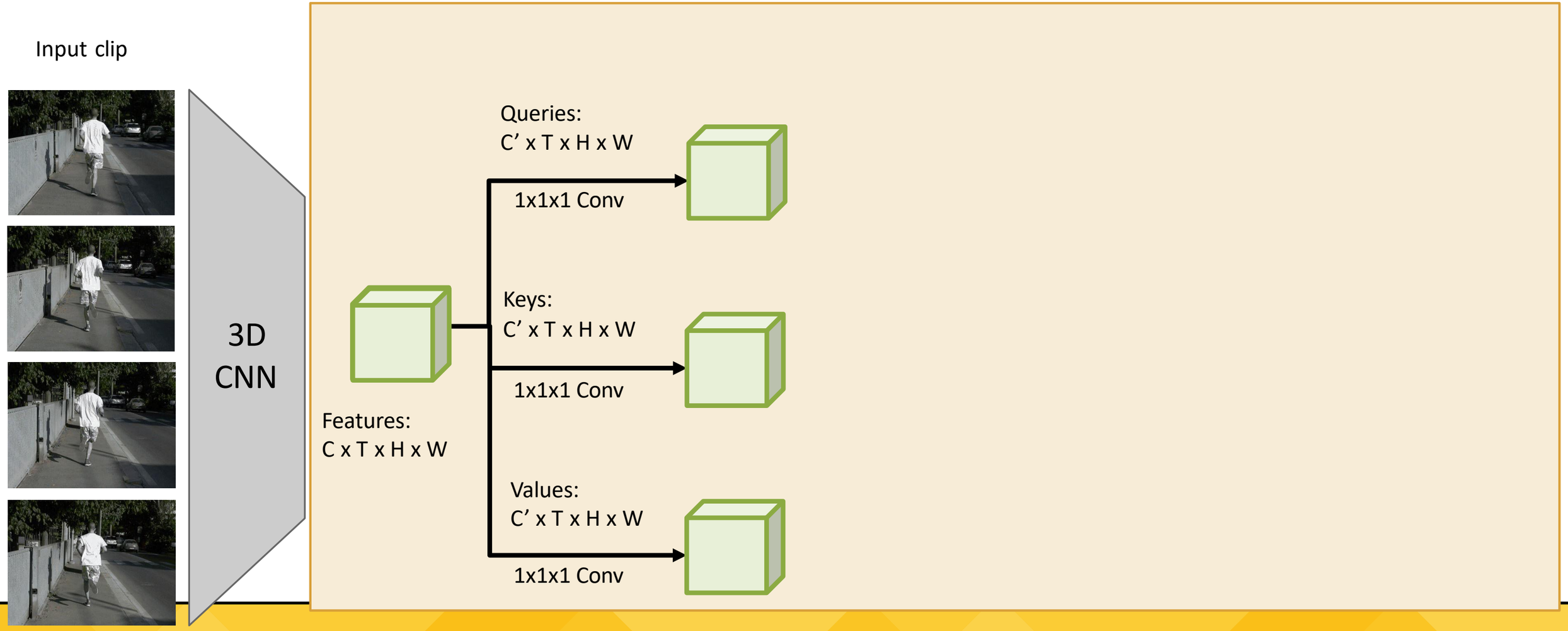


Attention based Video Understanding

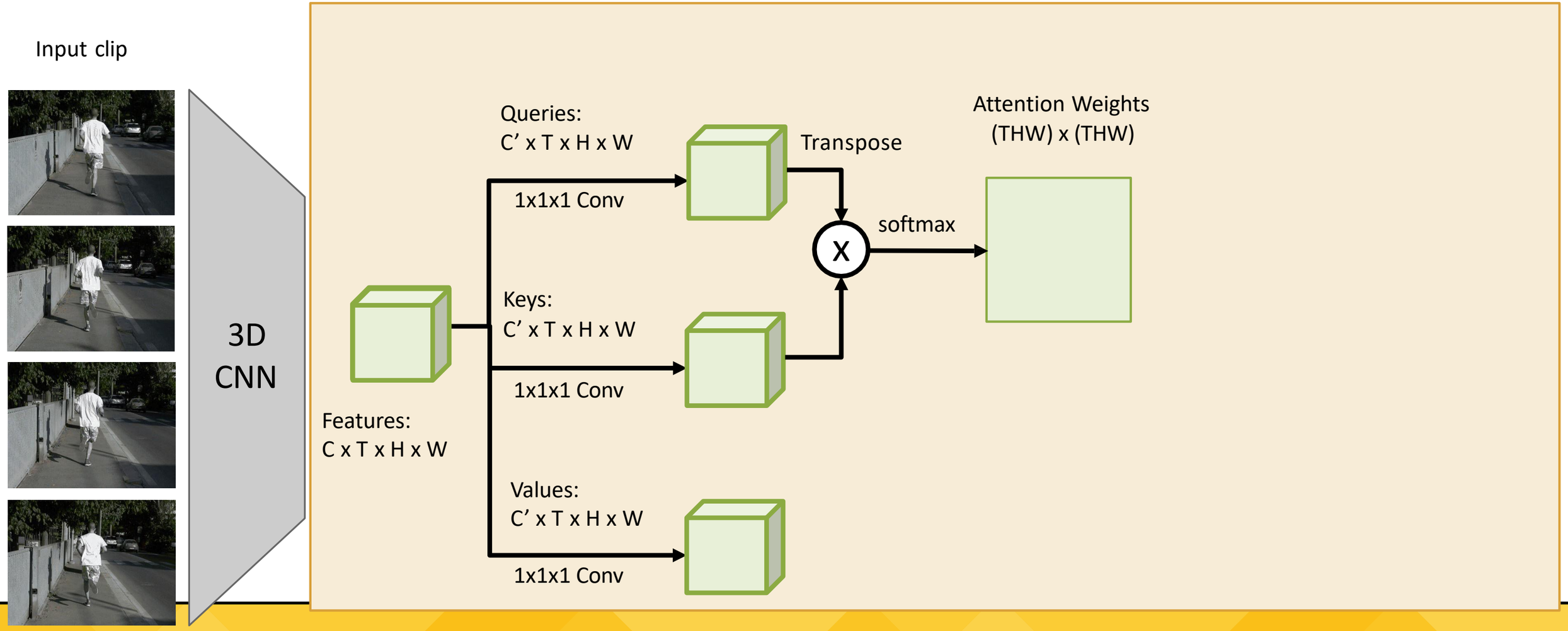
Spatio-Temporal Self-Attention (Nonlocal Block)



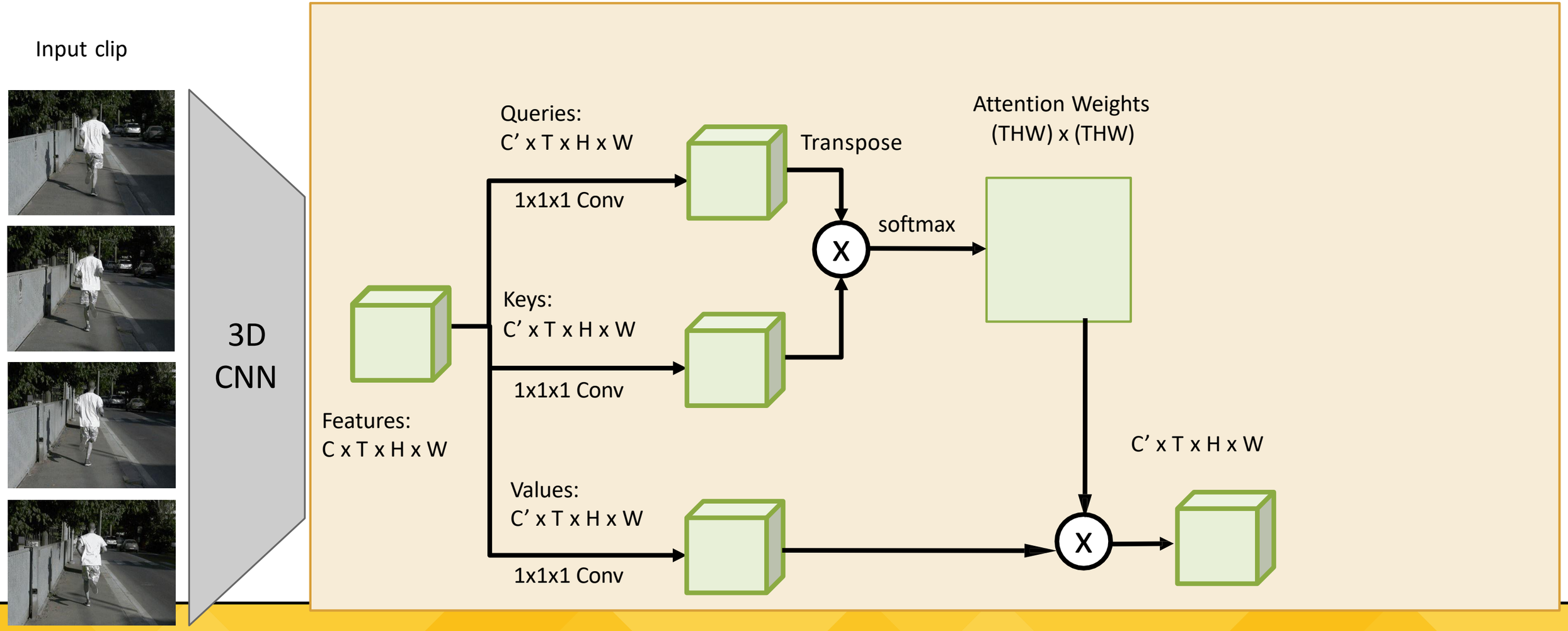
Spatio-Temporal Self-Attention (Nonlocal Block)



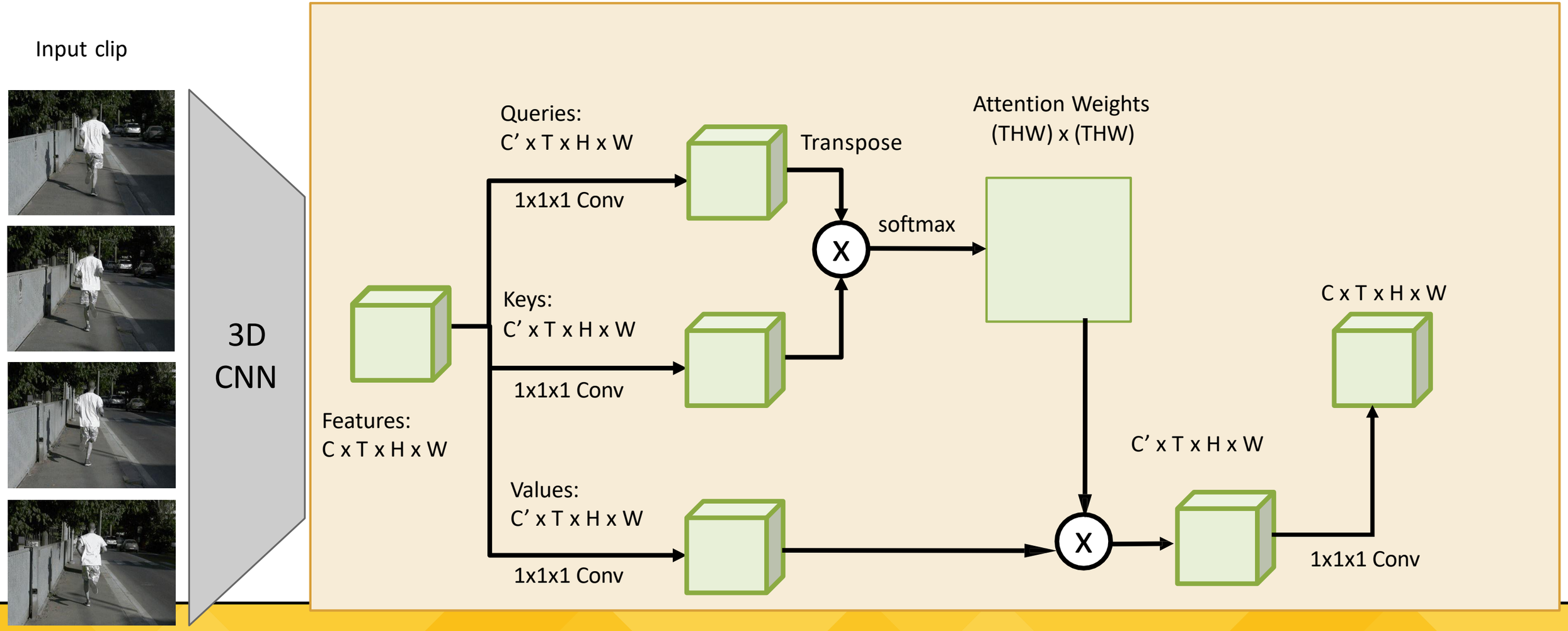
Spatio-Temporal Self-Attention (Nonlocal Block)



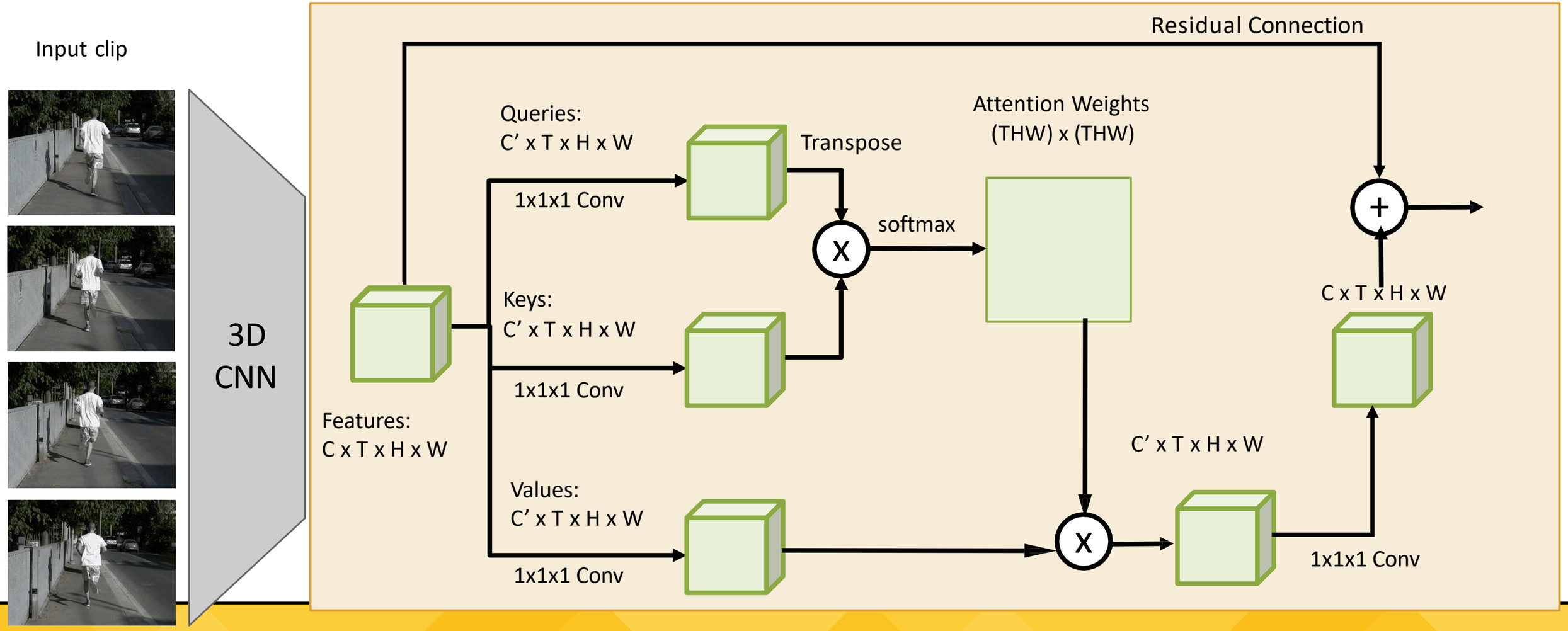
Spatio-Temporal Self-Attention (Nonlocal Block)



Spatio-Temporal Self-Attention (Nonlocal Block)



Spatio-Temporal Self-Attention (Nonlocal Block)

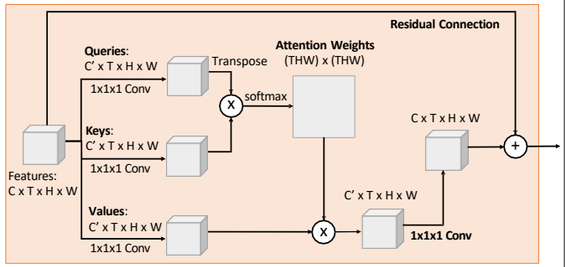


Spatio-Temporal Self-Attention (Nonlocal Block)

We can add nonlocal blocks into existing 3D CNN architectures.
But what is the best 3D CNN architecture?

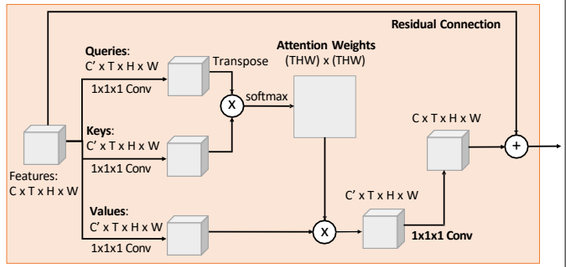
Input clip

3D CNN



Nonlocal Block

3D CNN



Nonlocal Block

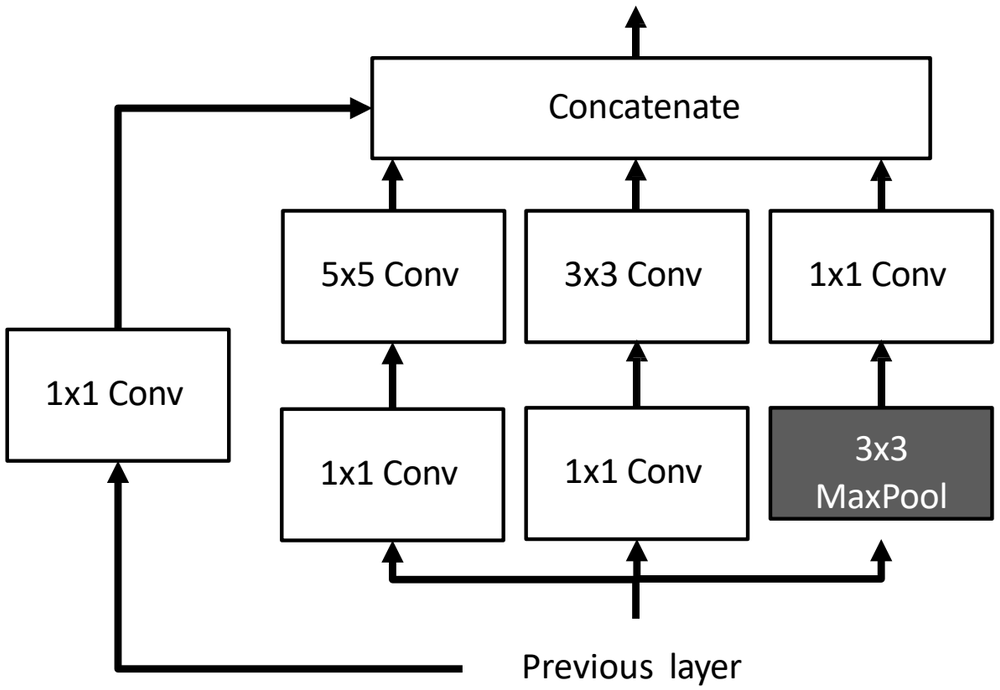
3D CNN

Running

Inflating 2D Networks to 3D (I3D)

- There has been a lot of work on architectures for images. Can we reuse image architectures for video?
- Idea: take a 2D CNN architecture.
- Replace each 2D Kh x Kw conv/pool layer with a 3D Kt x Kh x Kw version

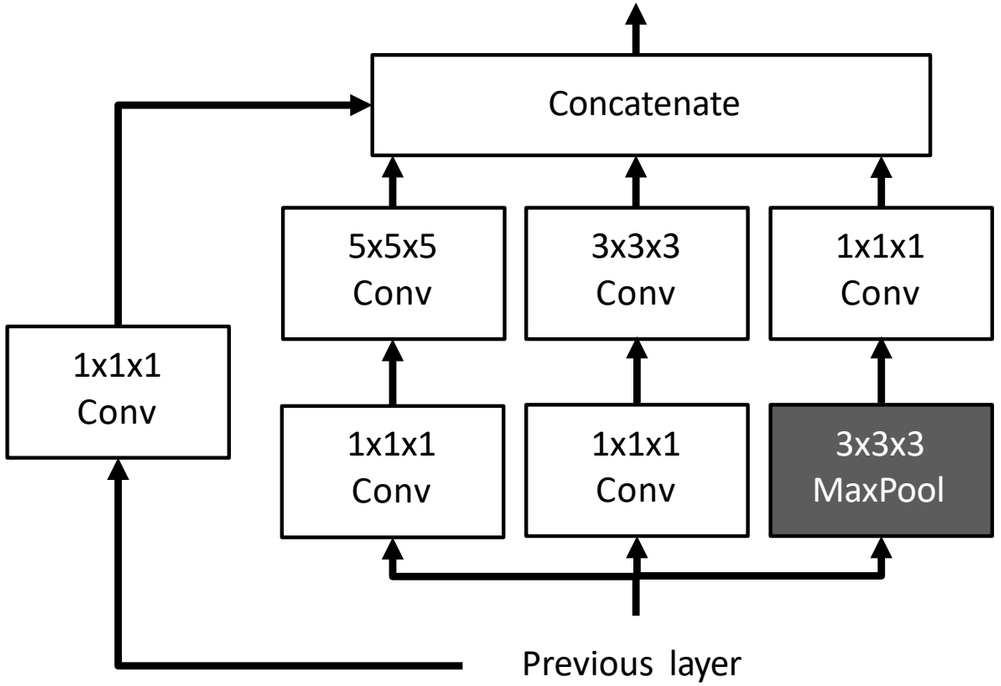
Inception Block: Original



Inflating 2D Networks to 3D (I3D)

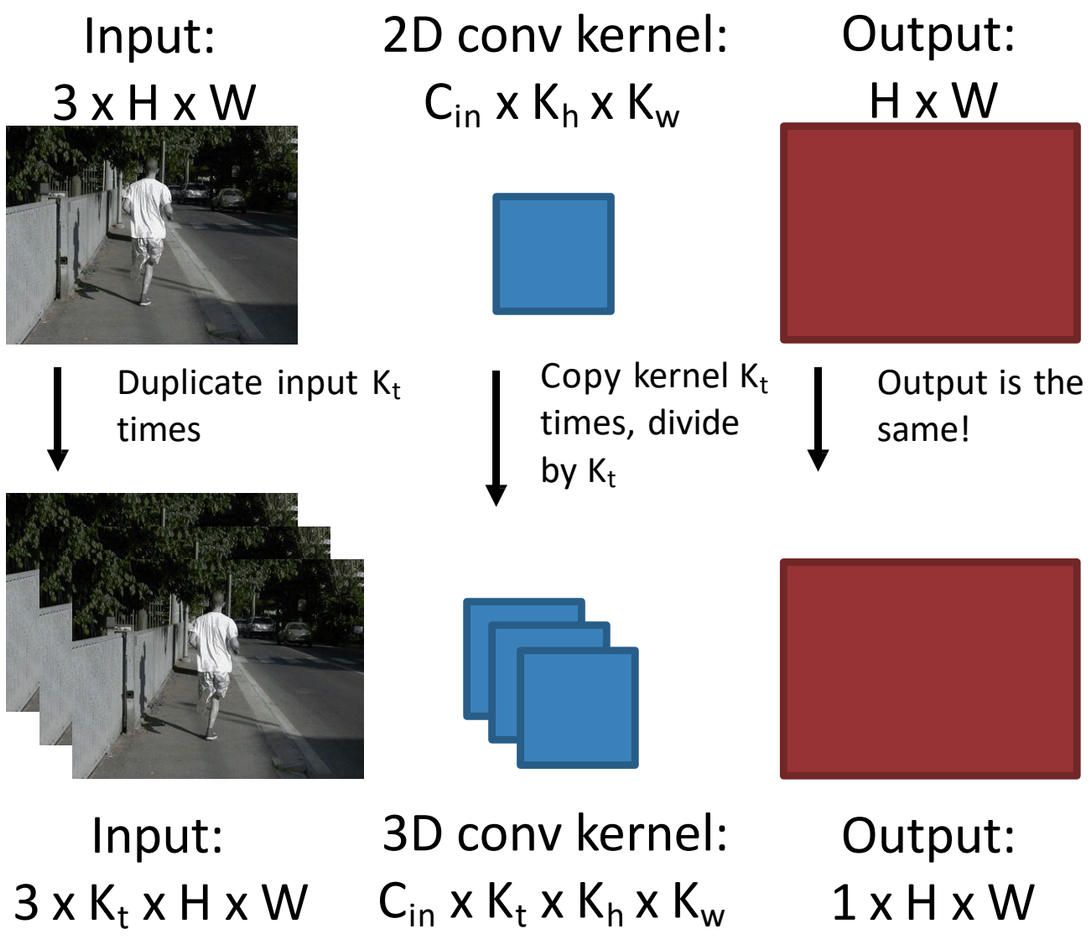
- There has been a lot of work on architectures for images. Can we reuse image architectures for video?
- Idea: take a 2D CNN architecture.
- Replace each 2D K_h x K_w conv/pool layer with a 3D K_t x K_h x K_w version

Inception Block: Inflated

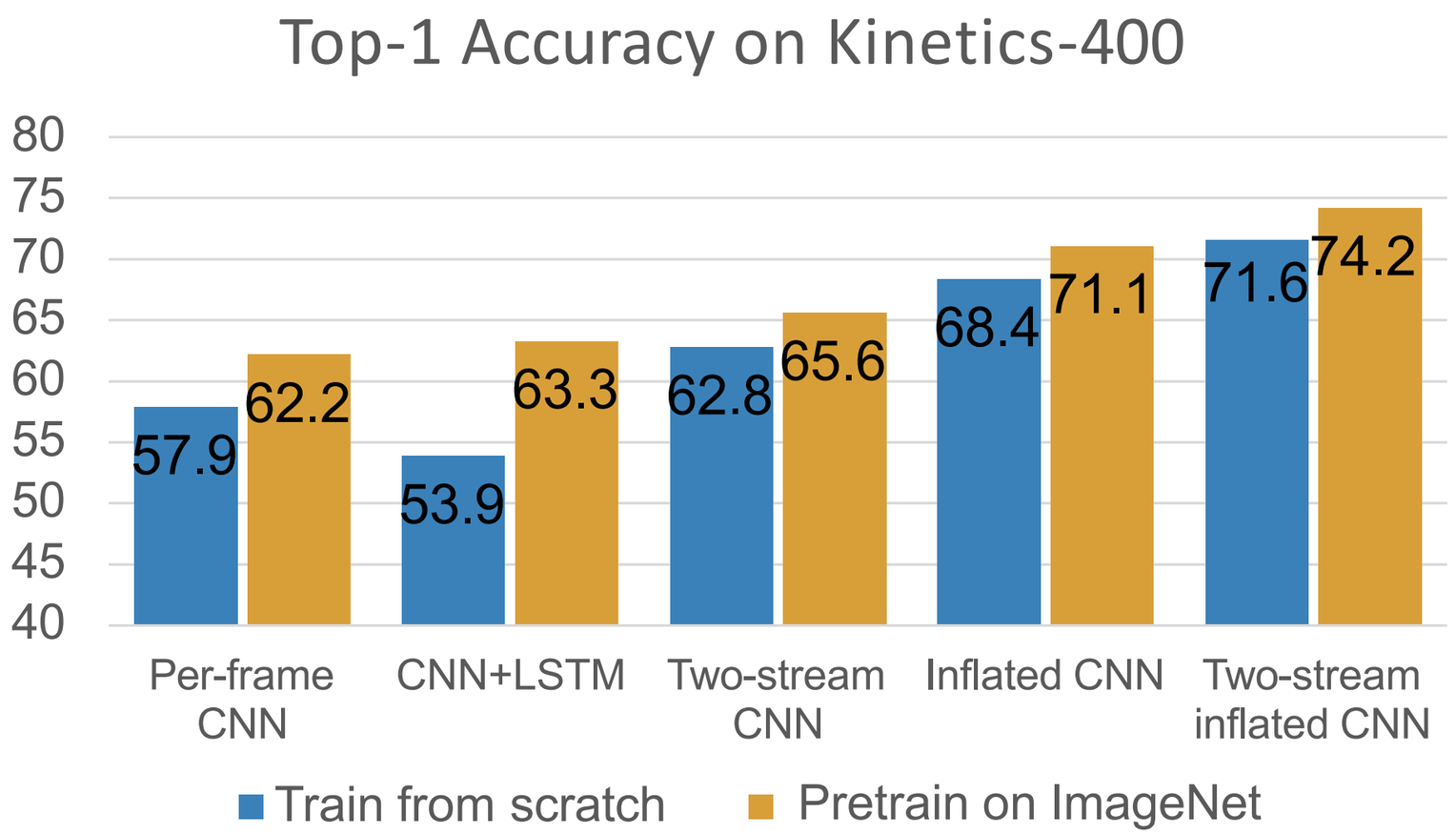


Inflating 2D Networks to 3D (I3D)

- There has been a lot of work on architectures for images. Can we reuse image architectures for video?
- Idea: take a 2D CNN architecture.
- Replace each 2D K_h x K_w conv/pool layer with a 3D K_t x K_h x K_w version
- Can use weights of 2D conv to initialize 3D conv: copy K_t times in space and divide by K_t
- This gives the same result as 2D conv given "constant" video input



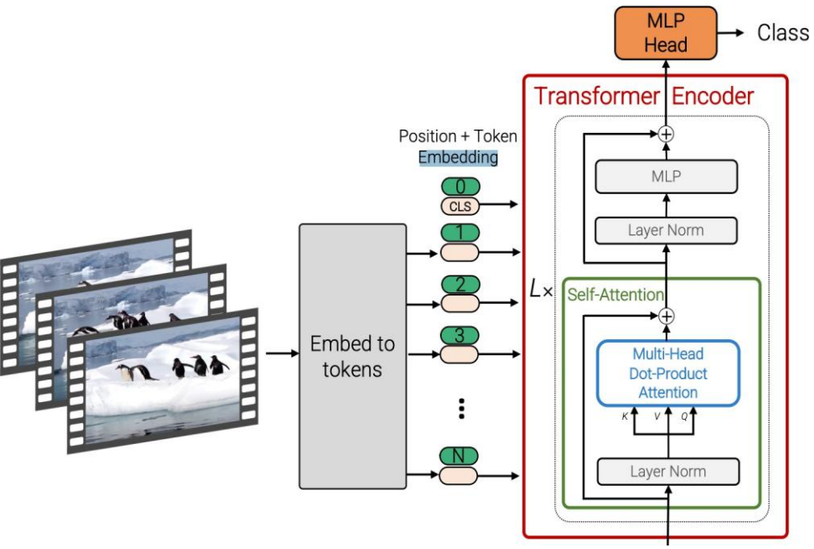
Inflating 2D Networks to 3D (I3D)



All using Inception CNN

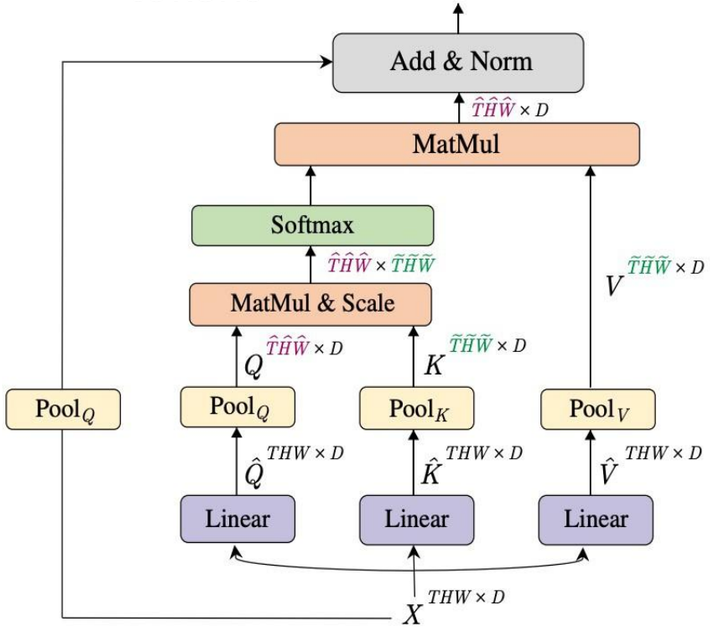
Vision Transformers for Video

Factorized attention: Attend over space / time



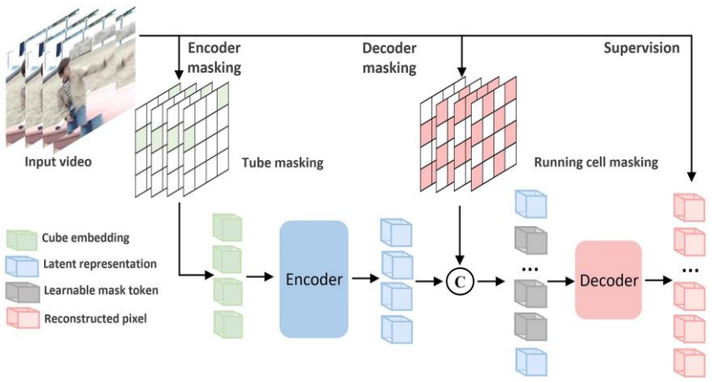
Bertasius et al, "Is Space-Time Attention All You Need for Video Understanding?", ICML 2021
Arnab et al, "ViViT: A Video Vision Transformer", ICCV 2021 Neimark et al, "Video Transformer Network", ICCV 2021

Pooling module: Reduce number of tokens



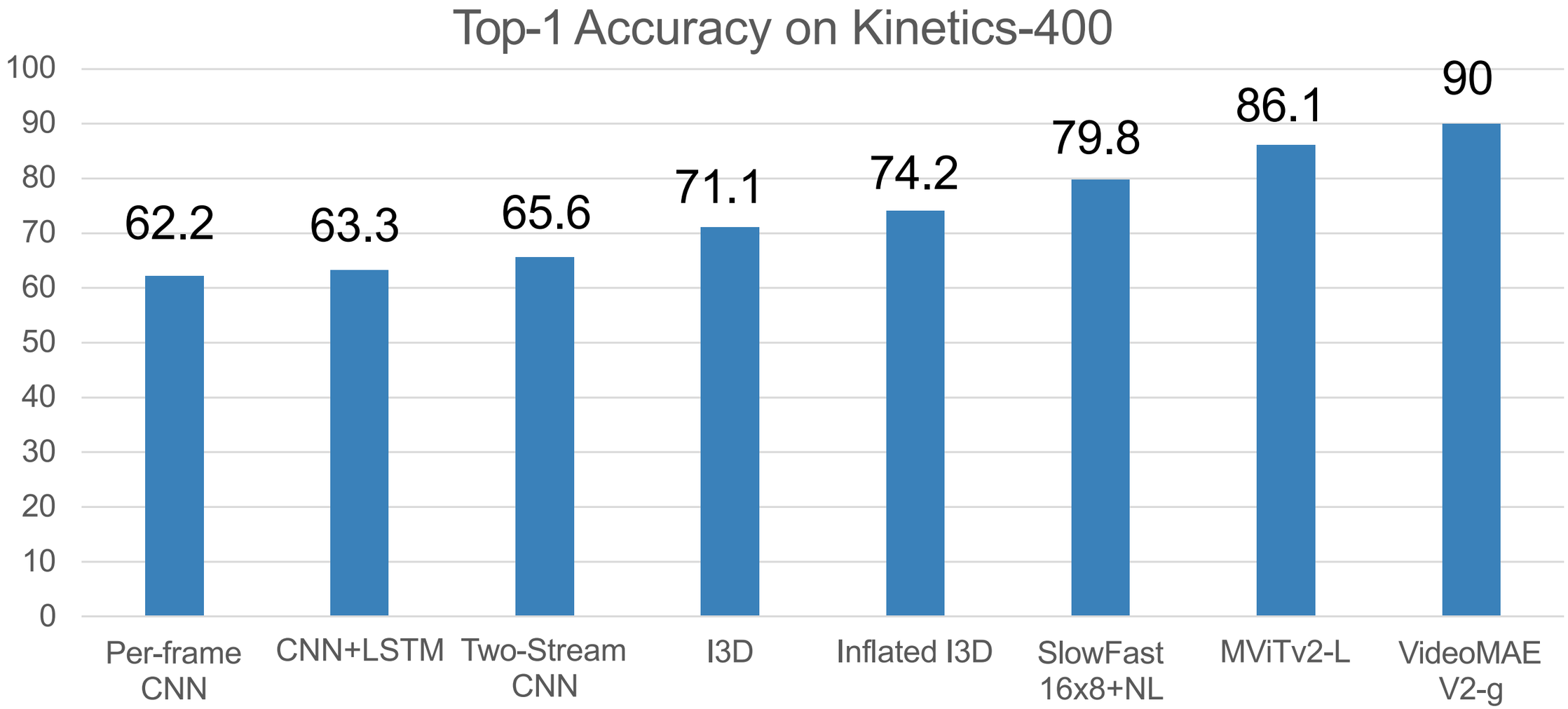
Fan et al, "Multiscale Vision Transformers", ICCV 2021 Li et al, "MViTv2: Improved Multiscale Vision Transformers for Classification and Detection", CVPR 2022

Video masked autoencoders: Efficient scalable pretraining



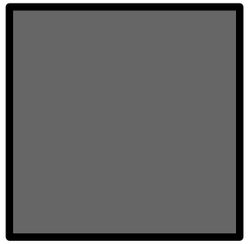
Wang et al. VideoMAE V2: Scaling Video Masked Autoencoders with Dual Making. CVPR 2023.
Tong et al. Video MAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training. NeurIPS 2022.
Feichtenhofer et al. Masked autoencoders as spatiotemporal learners. NeurIPS 2022.

Vision Transformers for Video



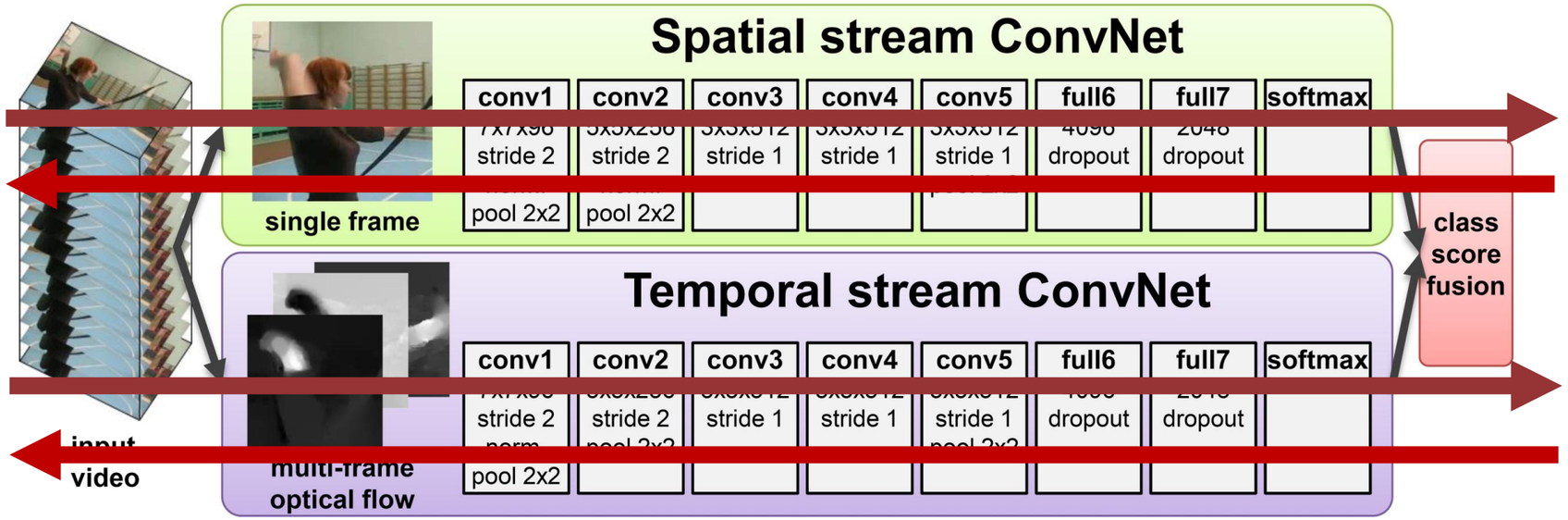
Visualizing Video Models

Image



Flow

Forward: Compute class score



”weightlifting ” score

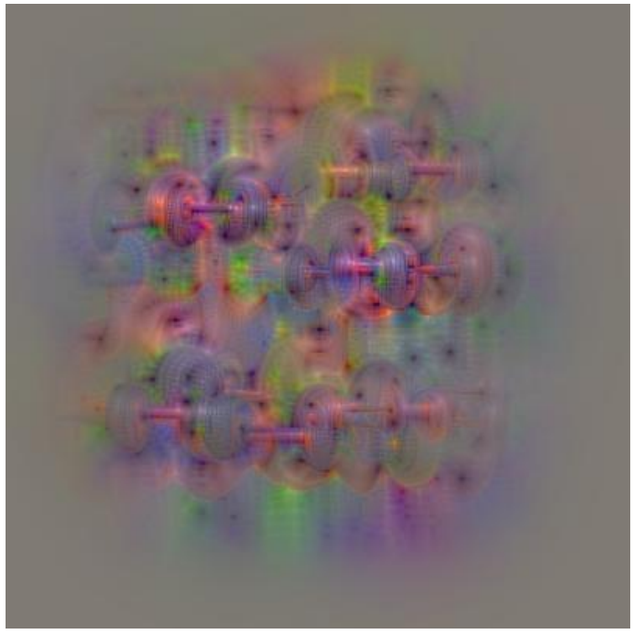
Backward: Compute gradient

Add a term to encourage spatially smooth flow; tune penalty to pick out “slow” vs “fast” motion

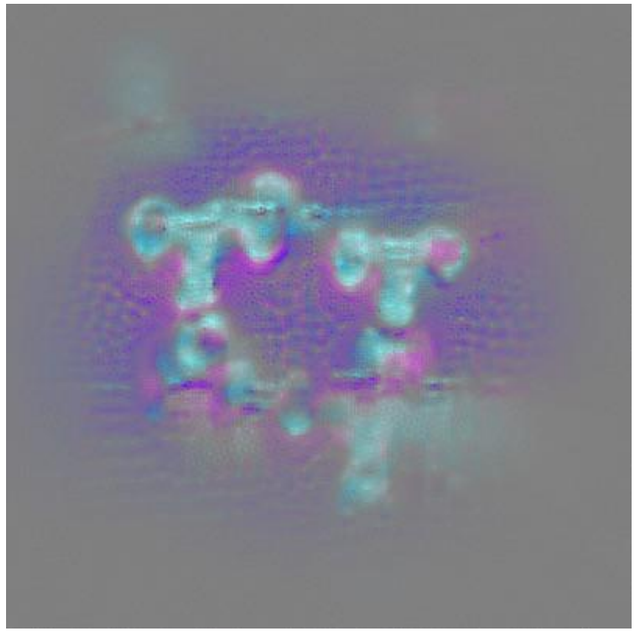
Figure credit: Simonyan and Zisserman, “Two-stream convolutional networks for action recognition in videos”, NeurIPS 2014
Feichtenhofer et al, “What have we learned from deep representations for action recognition?”, CVPR 2018
Feichtenhofer et al, “Deep insights into convolutional networks for video recognition?”, IJCV 2019.

Can you guess the action?

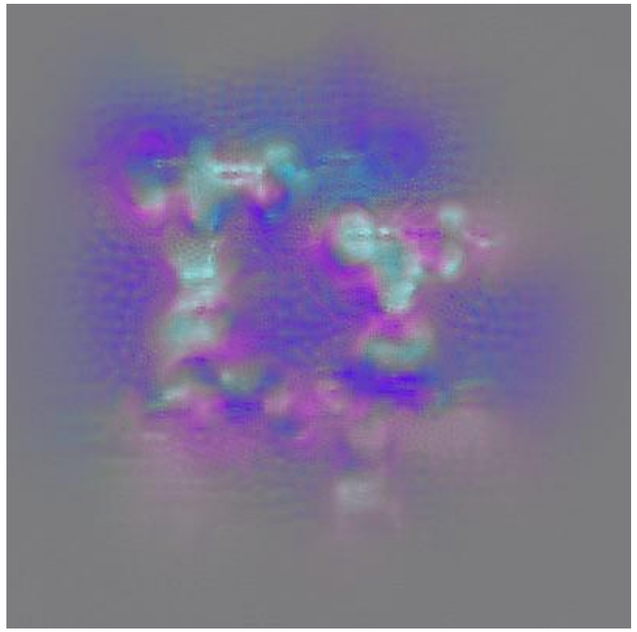
Appearance



“Slow” motion

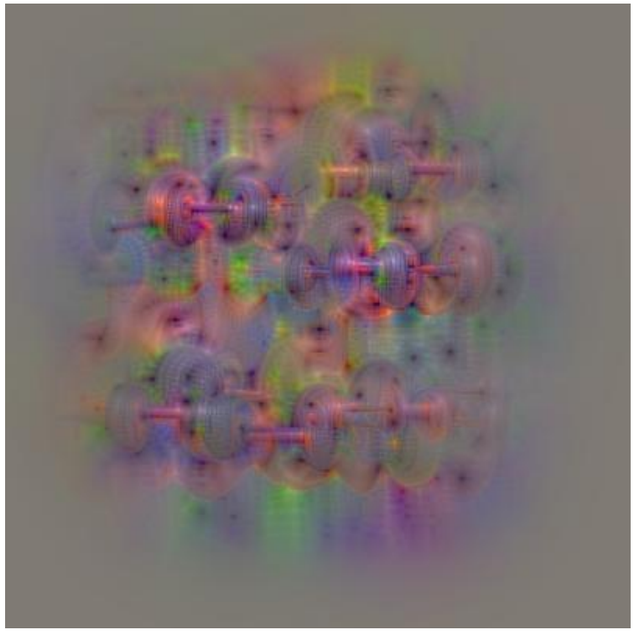


“Fast” motion

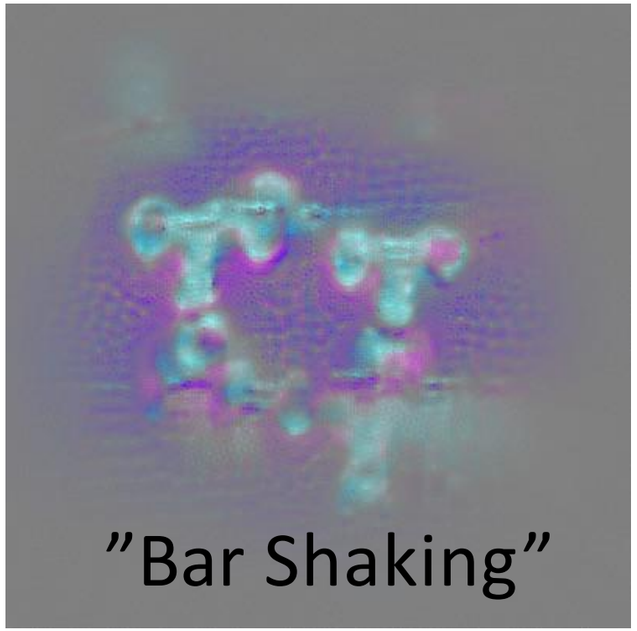


Can you guess the action? Weightlifting

Appearance



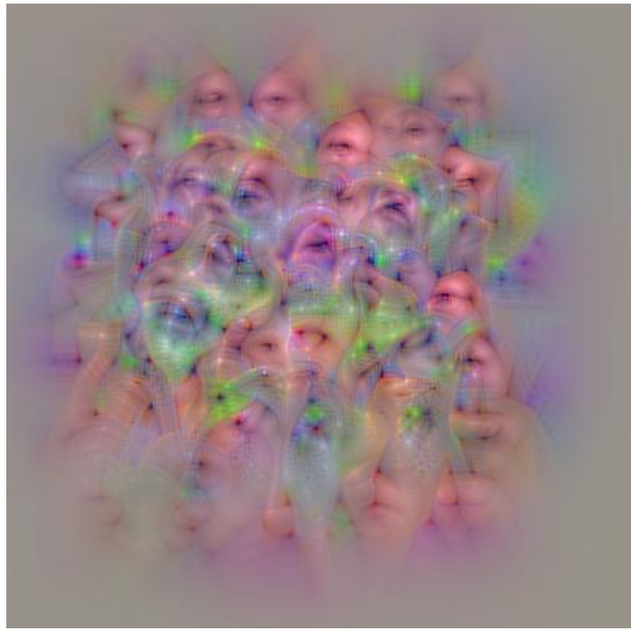
"Slow" motion



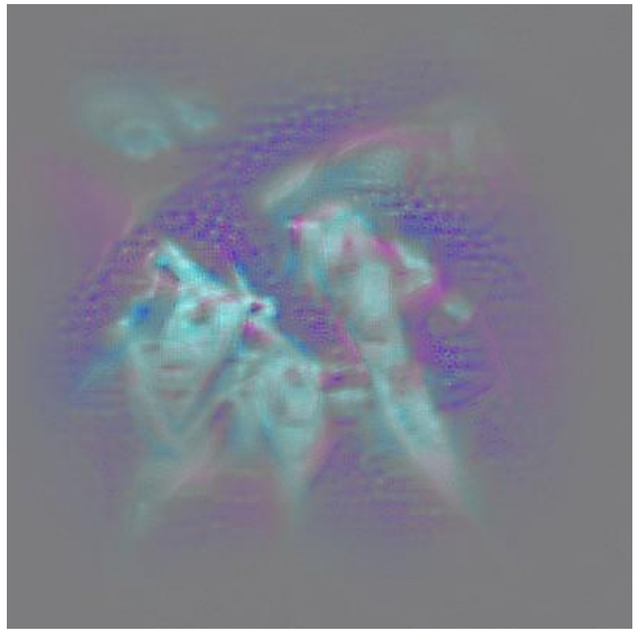
"Fast" motion

Can you guess the action? Apply Eye Makeup

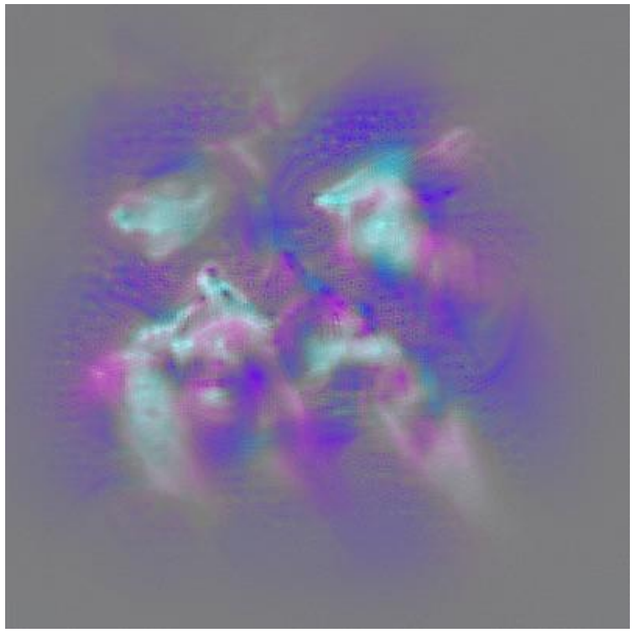
Appearance

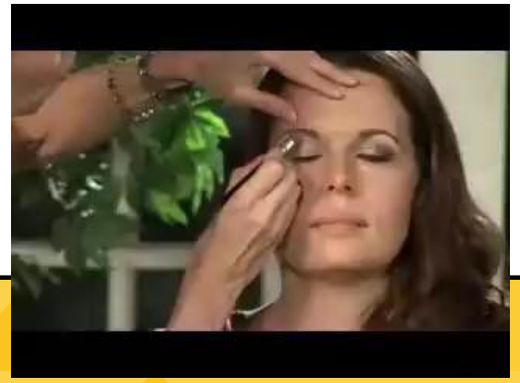


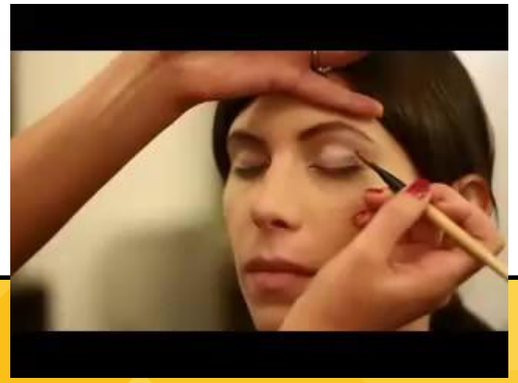
"Slow" motion



"Fast" motion



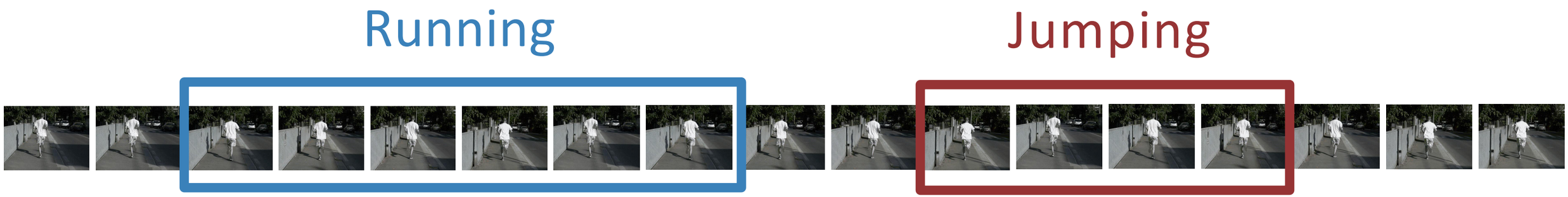




Application

Temporal Action Localization

- Given a long untrimmed video sequence, identify frames corresponding to different actions

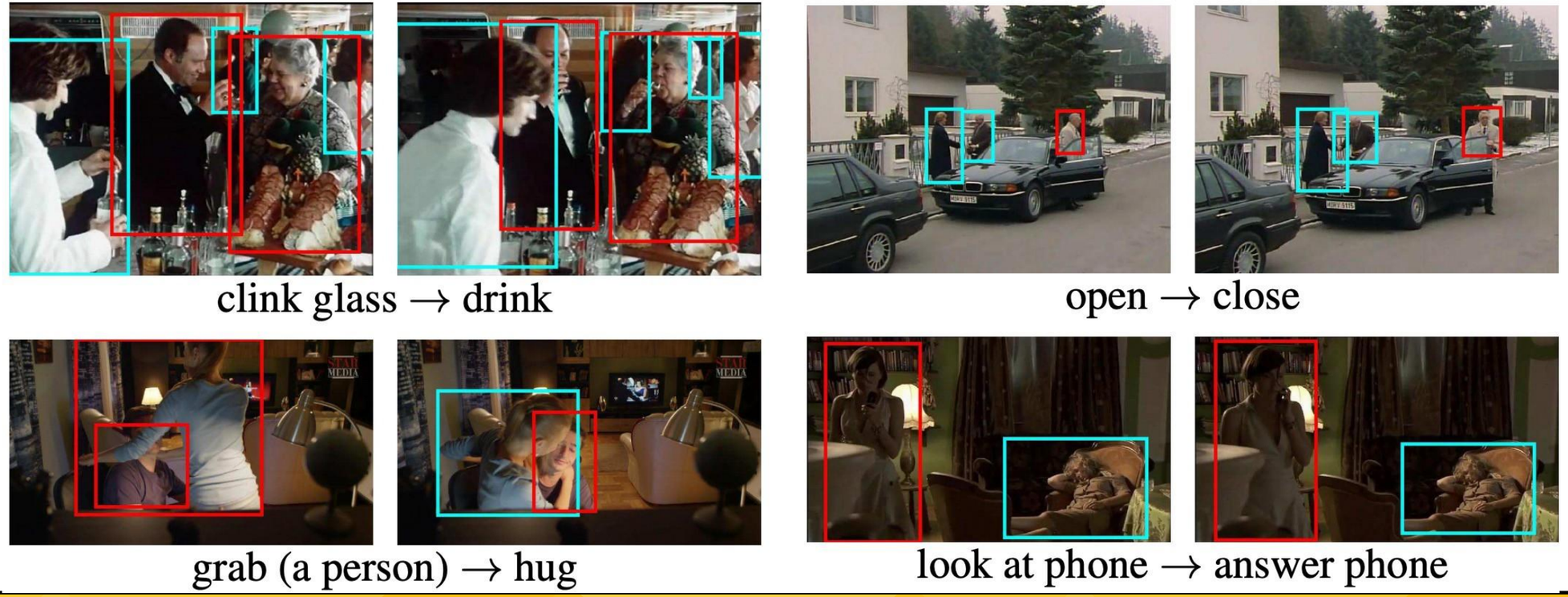


- Can use architecture similar to Faster R-CNN: first generate temporal proposals then classify

Chao et al, ” Rethinking the Faster R-CNN Architecture for Temporal Action Localization”, CVPR 2018

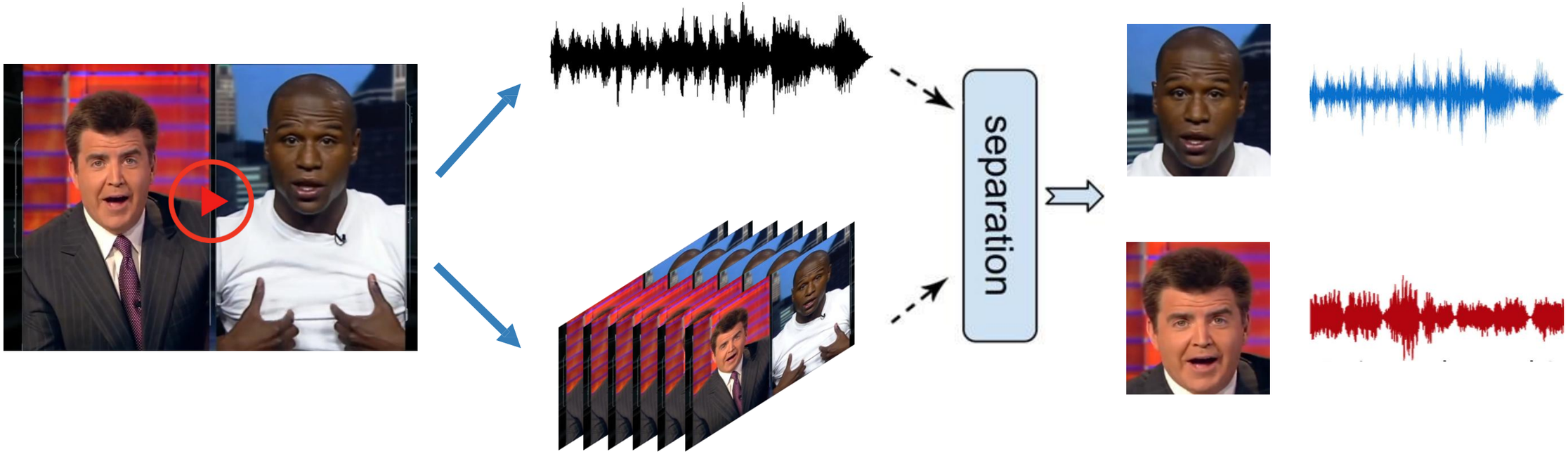
Spatio-Temporal Detection

- Given a long untrimmed video, detect all the people in both space and time and classify the activities they are performing. Some examples from AVA Dataset:



Gu et al, “AVA: A Video Dataset of Spatio-temporally Localized Atomic Visual Actions”, CVPR 2018

Visually-guided audio source separation



[Gao et al. ECCV 2018, Afouras et al. Interspeech'18, Gabby et al. Interspeech'18, Owens & Efros ECCV'18, Ephrat et al. SIGGRAPH'18, Zhao et al. ECCV 2018, Gao & Grauman ICCV 2019, Zhao et al. ICCV 2019, Xu et al. ICCV 2019, Gan et al. CVPR 2020, Gao et al. CVPR 2021, Tzinis et al. ECCV 2022, Chen et al. CVPR 2023]

Musical instruments source separation

- Train on 100,000 unlabeled multi-source video clips, then separate audio for novel video.

original video
(before separation)

object detections:
violin & flute

Gao & Grauman, Co-Separating Sounds of Visual Objects, ICCV 2019

Summary

- From 2D to 3D CNN for video understanding
- Optical Flow
- RNN based video understanding
- Attention based video understanding
- Applications

References

- Stanford CS231n